

```
# Loading libraries
```

```
import pandas as pd
import numpy as np
import re
import os
import warnings
import matplotlib.pyplot as plt
from google.colab import files
```

```
# Install plotnine
!pip install plotnine
from plotnine import *
```

```
warnings.filterwarnings('ignore')
```

```
# Display all columns in pandas DataFrame
pd.set_option('display.max_columns', None)
```

```
# Upload files to Colab
uploaded = files.upload()
```

```
Requirement already satisfied: plotnine in /usr/local/lib/python3.11/dist-packages (0.14.5)
Requirement already satisfied: matplotlib>=3.8.0 in /usr/local/lib/python3.11/dist-packages (from plotnine)
Requirement already satisfied: pandas>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from plotnine) (2.
Requirement already satisfied: mizani<=0.13.0 in /usr/local/lib/python3.11/dist-packages (from plotnine) (0
Requirement already satisfied: numpy>=1.23.5 in /usr/local/lib/python3.11/dist-packages (from plotnine) (2.
Requirement already satisfied: scipy>=1.8.0 in /usr/local/lib/python3.11/dist-packages (from plotnine) (1.1
Requirement already satisfied: statsmodels>=0.14.0 in /usr/local/lib/python3.11/dist-packages (from plotnin
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib>
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3.8.0
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplc
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=2.2.0-
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=2.2.
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.11/dist-packages (from statsmodels>=0
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2
```

 Choose Files 11 files

- **final_2021_clean.csv**(text/csv) - 586594 bytes, last modified: 4/9/2025 - 100% done
 - **OATH_Hearings_Division_Case_Status_NYPD ONLY (Jan 1 2021 to Aug 31 2022).csv**(text/csv) - 4991588 bytes, last modified: 3/31/2025 - 100% done
 - **rs_processed_clean2.csv**(text/csv) - 2076663 bytes, last modified: 7/30/2022 - 100% done
 - **first_set_updated.csv**(text/csv) - 4418659 bytes, last modified: 7/30/2022 - 100% done
 - **charges_clean2.csv**(text/csv) - 1721499 bytes, last modified: 7/30/2022 - 100% done
 - **disciplinary_history_clean.csv**(text/csv) - 645628 bytes, last modified: 7/30/2022 - 100% done
 - **ar_processed_clean.csv**(text/csv) - 6062327 bytes, last modified: 7/30/2022 - 100% done
 - **arrest_all.csv**(text/csv) - 2566070 bytes, last modified: 7/30/2022 - 100% done
 - **allegation_detail_clean.csv**(text/csv) - 822502 bytes, last modified: 7/30/2022 - 100% done
 - **disp_actions5(1).csv**(text/csv) - 135545 bytes, last modified: 3/31/2025 - 100% done
 - **personnel file(3).csv**(text/csv) - 3245732 bytes, last modified: 3/31/2025 - 100% done
- Saving final_2021_clean.csv to final_2021_clean (5).csv
 Saving OATH_Hearings_Division_Case_Status_NYPD ONLY (Jan 1 2021 to Aug 31 2022).csv to OATH_Hearings_Divisi
 Saving rs_processed_clean2.csv to rs_processed_clean2 (5).csv
 Saving first_set_updated.csv to first_set_updated (5).csv
 Saving charges_clean2.csv to charges_clean2 (5).csv
 Saving disciplinary_history_clean.csv to disciplinary_history_clean (5).csv
 Saving ar_processed_clean.csv to ar_processed_clean (5).csv
 Saving arrest_all.csv to arrest_all (5).csv
 Saving allegation_detail_clean.csv to allegation_detail_clean (5).csv
 Saving disp_actions5(1).csv to disp_actions5(1) (5).csv
 Saving personnel file(3).csv to personnel file(3) (5).csv

✓ Merge OATH_Hearings_Division_Case_Status_NYPD ONLY (Jan 1 2021 to Aug 31 2022).csv BY ticket number

```
# Reading in OATH_Hearings_Division_Case_Status_NYPD ONLY (Jan 1 2021 to Aug 31 2022).csv
oath = pd.read_csv("/content/OATH_Hearings_Division_Case_Status_NYPD ONLY (Jan 1 2021 to Aug 31 2022).csv")
oath.info()
```

```

22 Hearing Status          13528 non-null object
23 Hearing Result          11505 non-null object
24 Scheduled Hearing Location 10051 non-null object
25 Hearing Date            13522 non-null object
26 Hearing Time            13528 non-null object
27 Decision Location (Borough) 6998 non-null object
28 Decision Date          11480 non-null object
29 Total Violation Amount  13528 non-null object
30 Violation Details       13391 non-null object
31 Date Judgment Docketed  4678 non-null object
32 Respondent Address or Facility Number(For FDNY and DOB Tickets) 6417 non-null object
33 Penalty Imposed        13528 non-null object
34 Paid Amount            13528 non-null object
35 Additional Penalties or Late Fees 13528 non-null int64
36 Compliance Status      13528 non-null object
37 Violation Description   0 non-null float64
38 Charge #1: Code        10833 non-null object
39 Charge #1: Code Section 10827 non-null object
40 Charge #1: Code Description 10827 non-null object
41 Charge #1: Infraction Amount 13528 non-null object
42 Charge #2: Code        2172 non-null object
43 Charge #2: Code Section 2165 non-null object
44 Charge #2: Code Description 2165 non-null object
45 Charge #2: Infraction Amount 3759 non-null object
46 Charge #3: Code        56 non-null object
47 Charge #3: Code Section 45 non-null object
48 Charge #3: Code Description 45 non-null object
49 Charge #3: Infraction Amount 76 non-null object
50 Charge #4: Code        0 non-null float64
51 Charge #4: Code Section 0 non-null float64
52 Charge #4: Code Description 0 non-null float64
53 Charge #4: Infraction Amount 0 non-null float64
54 Charge #5: Code        0 non-null float64
55 Charge #5: Code Section 0 non-null float64
56 Charge #5: Code Description 0 non-null float64
57 Charge #5: Infraction Amount 0 non-null float64
58 Charge #6: Code        0 non-null float64
59 Charge #6: Code Section 0 non-null float64
60 Charge #6: Code Description 0 non-null float64
61 Charge #6: Infraction Amount 0 non-null float64
62 Charge #7: Code        0 non-null float64
63 Charge #7: Code Section 0 non-null float64
64 Charge #7: Code Description 0 non-null float64
65 Charge #7: Infraction Amount 0 non-null float64
66 Charge #8: Code        0 non-null float64
67 Charge #8: Code Section 0 non-null float64
68 Charge #8: Code Description 0 non-null float64
69 Charge #8: Infraction Amount 0 non-null float64
70 Charge #9: Code        0 non-null float64
71 Charge #9: Code Section 0 non-null float64
72 Charge #9: Code Description 0 non-null float64
73 Charge #9: Infraction Amount 0 non-null float64
74 Charge #10: Code       0 non-null float64
75 Charge #10: Code Section 0 non-null float64
76 Charge #10: Code Description 0 non-null float64
77 Charge #10: Infraction Amount 0 non-null float64
dtypes: float64(34), int64(2), object(42)
memory usage: 8.1+ MB
```

```
# Reading in 2021 clean citations (to merge into)
clean_2021 = pd.read_csv("/content/final_2021_clean.csv")
clean_2021.info()
```

```

↳ <class 'pandas.core.frame.DataFrame'>
RangeIndex: 4663 entries, 0 to 4662
Data columns (total 18 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Ticket_Number                             4663 non-null   float64
1   Complainants_First_Name                   4662 non-null   object
2   Complainants_Last_Name                    4663 non-null   object
3   Tax_Registry_Number_Tax_ID                4663 non-null   object
4   Agency                                    4663 non-null   object
5   Confidence_on_PO_Name                     4663 non-null   object
6   Confidence_on_Officer_Tax_ID               4663 non-null   object
7   Alternative_Officer_First_Name             4663 non-null   object
8   Alternative_Officer_Last_Name              4663 non-null   object
9   Alternative_Officer_Tax_ID                 4663 non-null   object
10  Verified_Name_w_Tax_ID                     4663 non-null   object
11  Last_Name_First_Name                       4663 non-null   object
12  If_Partial_Illegible_ID                    4663 non-null   object
13  Final_Officer_First_Name                   4663 non-null   object
14  Final_Officer_Last_Name                    4663 non-null   object
15  Final_Officer_Tax_ID                       4662 non-null   float64
16  Computer_Generated_YN                     4662 non-null   object
17  If_Computer_Generated                      4661 non-null   object
dtypes: float64(2), object(16)
memory usage: 655.9+ KB

```

```

# Rename 'Ticket Number' to 'Ticket_Number' to match the clean dataset
oath.rename(columns={'Ticket Number': 'Ticket_Number'}, inplace=True)

```

```

# check for missing values in (Ticket_Number), is clean, just checking
clean_2021['Ticket_Number'].isnull().sum()

```

```

# convert 'Ticket_Number' from float64 to int64 to match OATH D type
clean_2021['Ticket_Number'] = clean_2021['Ticket_Number'].astype('int64')

```

```

# Performing OATH to draft_2021_clean initial merge
merged_df = pd.merge(
    clean_2021,
    oath,
    on='Ticket_Number', # now both datasets have the same column name
    how='left'
)

```

```

# Checking Oath merge
merged_df.info()

```



```

60 Charge #2: Code Section      007 non-null object
61 Charge #2: Code Description  607 non-null object
62 Charge #2: Infraction Amount 1186 non-null object
63 Charge #3: Code             25 non-null object
64 Charge #3: Code Section      16 non-null object
65 Charge #3: Code Description  16 non-null object
66 Charge #3: Infraction Amount 36 non-null object
67 Charge #4: Code             0 non-null float64
68 Charge #4: Code Section      0 non-null float64
69 Charge #4: Code Description  0 non-null float64
70 Charge #4: Infraction Amount 0 non-null float64
71 Charge #5: Code             0 non-null float64
72 Charge #5: Code Section      0 non-null float64
73 Charge #5: Code Description  0 non-null float64
74 Charge #5: Infraction Amount 0 non-null float64
75 Charge #6: Code             0 non-null float64
76 Charge #6: Code Section      0 non-null float64
77 Charge #6: Code Description  0 non-null float64
78 Charge #6: Infraction Amount 0 non-null float64
79 Charge #7: Code             0 non-null float64
80 Charge #7: Code Section      0 non-null float64
81 Charge #7: Code Description  0 non-null float64
82 Charge #7: Infraction Amount 0 non-null float64
83 Charge #8: Code             0 non-null float64
84 Charge #8: Code Section      0 non-null float64
85 Charge #8: Code Description  0 non-null float64
86 Charge #8: Infraction Amount 0 non-null float64
87 Charge #9: Code             0 non-null float64
88 Charge #9: Code Section      0 non-null float64
89 Charge #9: Code Description  0 non-null float64
90 Charge #9: Infraction Amount 0 non-null float64
91 Charge #10: Code            0 non-null float64
92 Charge #10: Code Section     0 non-null float64
93 Charge #10: Code Description 0 non-null float64
94 Charge #10: Infraction Amount 0 non-null float64
dtypes: float64(35), int64(2), object(58)
memory usage: 3.4+ MB

```

```

# Dropping all of the columns with 0 values
merged_df = merged_df.dropna(axis=1, how='all')
merged_df.info()

```

```

9  Alternative_Officer_Tax_ID      4663 non-null object
10 Verified_Name_w_Tax_ID         4663 non-null object
11 Last_Name_First_Name           4663 non-null object
12 If_Partial_Illegible_ID        4663 non-null object
13 Final_Officer_First_Name        4663 non-null object
14 Final_Officer_Last_Name         4663 non-null object
15 Final_Officer_Tax_ID           4662 non-null float64
16 Computer_Generated_YN          4662 non-null object
17 If_Computer_Generated           4661 non-null object
18 Violation Date                 4663 non-null object
19 Violation Time                 4663 non-null object
20 Issuing Agency                 4663 non-null object
21 Respondent First Name          4550 non-null object
22 Respondent Last Name           4660 non-null object
23 Balance Due                    4663 non-null object
24 Violation Location (Borough)    4663 non-null object
25 Violation Location (Block No.) 1291 non-null float64

```

```

40 scheduled hearing location      5550 non-null object
41 Hearing Date                     4663 non-null object
42 Hearing Time                     4663 non-null object
43 Decision Location (Borough)     2627 non-null object
44 Decision Date                   3815 non-null object
45 Total Violation Amount          4663 non-null object
46 Violation Details               4645 non-null object
47 Date Judgment Docketed         1263 non-null object
48 Respondent Address or Facility Number(For FDNY and DOB Tickets) 2101 non-null object
49 Penalty Imposed                 4663 non-null object
50 Paid Amount                     4663 non-null object
51 Additional Penalties or Late Fees 4663 non-null int64
52 Compliance Status              4663 non-null object
53 Charge #1: Code                 3863 non-null object
54 Charge #1: Code Section         3859 non-null object
55 Charge #1: Code Description     3859 non-null object
56 Charge #1: Infraction Amount    4663 non-null object
57 Charge #2: Code                 611 non-null object
58 Charge #2: Code Section         607 non-null object
59 Charge #2: Code Description     607 non-null object
60 Charge #2: Infraction Amount    1186 non-null object
61 Charge #3: Code                 25 non-null object
62 Charge #3: Code Section         16 non-null object
63 Charge #3: Code Description     16 non-null object
64 Charge #3: Infraction Amount    36 non-null object
dtypes: float64(5), int64(2), object(58)
memory usage: 2.3+ MB

```

```

# Quality Control: Check for tickets in clean_2021 that didn't find a match in the OATH dataset
total_tickets = len(clean_2021)

```

```

# Number of matched tickets by Ticket_Number
matched_count = merged_df['Ticket_Number'].notnull().sum()

```

```

# Number of unmatched tickets by Ticket_Number
unmatched_count = merged_df['Ticket_Number'].isnull().sum()

```

```

# Matched percentage
matched_pct = matched_count / total_tickets * 100

```

```

# Unmatched percentage
unmatched_pct = unmatched_count / total_tickets * 100

```

```

# Report
print(f"Total tickets: {total_tickets}")
print(f"Matched tickets: {matched_count} ({matched_pct:.2f}%)" )
print(f"Unmatched tickets: {unmatched_count} ({unmatched_pct:.2f}%)" )

```

```

# Retained all ticket_number from original clean_2021

```

```

🔗 Total tickets: 4663
   Matched tickets: 4663 (100.00%)
   Unmatched tickets: 0 (0.00%)

```

```

# Fixing "Final_Officer_Tax_ID" values to not be floats
merged_df["Final_Officer_Tax_ID"] = merged_df["Final_Officer_Tax_ID"].apply(
    lambda x: str(int(float(x))) if pd.notnull(x) and str(x).replace(".", "").isdigit() else str(x)
)

```

✓ Merging all 10 officer demographic datasets to draft_2021_clean by TAXNUM

1. personnel file(3).csv :

- taxnum
- command
- dept_recognitions

- rank
- shield
- appt_date
- name (lastname, firstname)
- race

2. disp_actions5(1).csv :

- taxnum
- command
- charge (description)
- casenum
- disposition
- penalty
- X8

3. training_processed_clean.csv :

- taxnum
- training_date
- training_name (training location name)

4. rs_processsed_clean2.csv :

- taxnum
- date
- shield_num
- title

5. first_set_updates.csv :

- tax_num
- row_num
- name (lastname, firstname)
- first_name
- lats_name (variable typo)
- rank
- command
- shield
- appt_date
- total_arrests
- dept_recognitions
- ethnicity
- in_sample
- same_as_web_app
- arrest count dif
- award count dif
- notes

6. disciplinary_history_clean.csv :

- taxnum
- date
- charge_count
- allegation_count

7. charges_clean2.csv

- taxnum
- case_num
- charge
- case_value
- disposition

8. arrest_all.csv

- taxnum
- arrest_count
- arrest_type

9. ar_processed_clean

- taxnum
- date
- award (description)

10. allegation_detail_clean.csv

- taxnum
- case_num
- allegation
- case_value
- recommendation

▼ 1. personnel file(3).csv Merge

```
# Read in personnel file(3).csv
personnel_file = pd.read_csv("/content/personnel file(3).csv")
personnel_file.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34221 entries, 0 to 34220
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   command               34221 non-null  object
1   dept_recognitions     34221 non-null  int64
2   rank                  34221 non-null  object
3   shield                31936 non-null  float64
4   appt_date             34221 non-null  object
5   total_arrests         34221 non-null  int64
6   name                  34221 non-null  object
7   taxnum                34221 non-null  int64
8   race                  34218 non-null  object
dtypes: float64(1), int64(3), object(5)
memory usage: 2.3+ MB
```

```
# Ensure both fields are strings
personnel_file["taxnum"] = personnel_file["taxnum"].astype(str)
merged_df["Final_Officer_Tax_ID"] = merged_df["Final_Officer_Tax_ID"].astype(str)

# Correct match mask (check if Final_Officer_Tax_ID exists in personnel_file["taxnum"])
matched_tax_ids = merged_df["Final_Officer_Tax_ID"].isin(personnel_file["taxnum"])

# Count matches
total_rows = len(merged_df)
num_matches = matched_tax_ids.sum()
num_unmatched = total_rows - num_matches
match_percentage = round((num_matches / total_rows) * 100, 2)

# Print results
print(f"Total Rows in merged_df: {total_rows},")
```

```
print(f"Matched Tax IDs: {num_matches:,}")
print(f"Unmatched Tax IDs: {num_unmatched:,}")
print(f"Match Percentage: {match_percentage}%")
```

```
➦ Total Rows in merged_df: 4,663
  Matched Tax IDs: 4,513
  Unmatched Tax IDs: 150
  Match Percentage: 96.78%
```

```
# Ensure both columns are strings
personnel_file["taxnum"] = personnel_file["taxnum"].astype(str)
merged_df["Final_Officer_Tax_ID"] = merged_df["Final_Officer_Tax_ID"].astype(str)

# Get only personnel rows that match Final_Officer_Tax_IDs
matched_personnel = personnel_file[personnel_file["taxnum"].isin(merged_df["Final_Officer_Tax_ID"])]

# Count how many times each taxnum appears in matched_personnel
taxnum_counts = matched_personnel["taxnum"].value_counts()

# Get duplicates: taxnums appearing more than once
duplicate_taxnums = taxnum_counts[taxnum_counts > 1]

# Print results
print(f"Total matched taxnums: {matched_personnel['taxnum'].nunique()}")
print(f"Total matching rows in personnel_file: {len(matched_personnel)}")
print(f"Number of duplicate taxnums: {len(duplicate_taxnums)}")

# Display duplicate taxnums if they exist
if not duplicate_taxnums.empty:
    print("\nDuplicate matched taxnums (taxnum: count):")
    print(duplicate_taxnums)
else:
    print("No duplicate matched taxnums found.")
```

```
➦ Total matched taxnums: 2311
  Total matching rows in personnel_file: 2311
  Number of duplicate taxnums: 0
  No duplicate matched taxnums found.
```

```
# Merging both files; retain ids
merged_df["Final_Officer_Tax_ID"] = merged_df["Final_Officer_Tax_ID"].astype(str)
personnel_file["taxnum"] = personnel_file["taxnum"].astype(str)

# Perform left join to keep all Final_Officer_Tax_IDs from merged_df
merged_with_personnel = merged_df.merge(
    personnel_file,
    how="left",
    left_on="Final_Officer_Tax_ID",
    right_on="taxnum"
)

# How many matched (non-null names from personnel file)?
num_matched = merged_with_personnel["name"].notna().sum()
num_total = len(merged_with_personnel)
print(f"Matched Officers: {num_matched:,}")
print(f"Unmatched Officers: {num_total - num_matched:,}")
```

```
➦ Matched Officers: 4,513
  Unmatched Officers: 150
```

```
merged_with_personnel.head()
```




	Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency	Con
0	190851430		PO	Soreca	953430	NYPD
1	196184708	.		Ennis	961743	NYPD
2	196241834	.		Wynno	961465	NYPD
3	196242292	.		Keniesky	964085	NYPD
4	196244840	.		Sinto	964778	NYPD

```
# Check how many entries have multiple RS dates
multiple_pf_rank = merged_with_personnel["rank"].apply(lambda x: isinstance(x, list) and len(x) > 1)
```

```
# Count how many rows this applies to
print("Officers with multiple RS_dates:", multiple_pf_rank.sum())
```

```
# Optional: view them
merged_with_personnel[multiple_pf_rank][["Final_Officer_Tax_ID", "rank", "appt_date"]].head()
```



Officers with multiple RS_dates: 0

Final_Officer_Tax_ID	rank	appt_date
----------------------	------	-----------



```
# Drop redundant "taxnum" column
merged_with_personnel = merged_with_personnel.drop(columns=["taxnum"])
```

```
# Still retained all 4663 ticket numbers
merged_with_personnel.info()
```



```

26 violation_location (Lot No.) 1291 non-null float64
27 Violation Location (House #) 1734 non-null object
28 Violation Location (Street Name) 4647 non-null object
29 Violation Location (City) 4303 non-null object
30 Violation Location (Zip Code) 1596 non-null float64
31 Violation Location (State Name) 4247 non-null object
32 Respondent Address (Borough) 4663 non-null object
33 Respondent Address (House #) 4534 non-null object
34 Respondent Address (Street Name) 4585 non-null object
35 Respondent Address (City) 3815 non-null object
36 Respondent Address (Zip Code) 3414 non-null float64
37 Respondent Address (State Name) 3798 non-null object
38 Hearing Status 4663 non-null object
39 Hearing Result 3819 non-null object
40 Scheduled Hearing Location 3330 non-null object
41 Hearing Date 4663 non-null object
42 Hearing Time 4663 non-null object
43 Decision Location (Borough) 2627 non-null object
44 Decision Date 3815 non-null object
45 Total Violation Amount 4663 non-null object
46 Violation Details 4645 non-null object
47 Date Judgment Docketed 1263 non-null object
48 Respondent Address or Facility Number(For FDNY and DOB Tickets) 2101 non-null object
49 Penalty Imposed 4663 non-null object
50 Paid Amount 4663 non-null object
51 Additional Penalties or Late Fees 4663 non-null int64
52 Compliance Status 4663 non-null object
53 Charge #1: Code 3863 non-null object
54 Charge #1: Code Section 3859 non-null object
55 Charge #1: Code Description 3859 non-null object
56 Charge #1: Infraction Amount 4663 non-null object
57 Charge #2: Code 611 non-null object
58 Charge #2: Code Section 607 non-null object
59 Charge #2: Code Description 607 non-null object
60 Charge #2: Infraction Amount 1186 non-null object
61 Charge #3: Code 25 non-null object
62 Charge #3: Code Section 16 non-null object
63 Charge #3: Code Description 16 non-null object
64 Charge #3: Infraction Amount 36 non-null object
65 command 4513 non-null object
66 dept_recognitions 4513 non-null float64
67 rank 4513 non-null object
68 shield 4510 non-null float64
69 appt_date 4513 non-null object
70 total_arrests 4513 non-null float64
71 name 4513 non-null object
72 race 4513 non-null object
dtypes: float64(7), int64(2), object(64)
memory usage: 2.6+ MB

```

2. disp_actions5(1).csv Merge

- There are multiple taxnums for one officer, meaning that a singular officer has multiple disciplinary actions under taxnum
- To account for this, merge. An officer (singular tax id) will have a disp_charge_1, disp_casnum1, disp_disposition_1, etc. for a each disciplinary action

```

# Read in disp_actions5(1).csv
disp_actions = pd.read_csv("/content/disp_actions5(1).csv")
disp_actions.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 588 entries, 0 to 587
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   taxnum          588 non-null    int64
1   command         587 non-null    object
2   charge          587 non-null    object
3   casenum         587 non-null    object
4   disposition     587 non-null    object

```

```

5   penalty      587 non-null    object
6   X8           588 non-null    object
dtypes: int64(1), object(6)
memory usage: 32.3+ KB

```

```

matches = disp_actions["taxnum"].isin(merged_with_personnel["Final_Officer_Tax_ID"])
matches.sum()

```

```
np.int64(0)
```

```
matches.sum()
```

```
np.int64(0)
```

```

# Checking for duplicates: Officers with multiple disciplinary charges
# Step 1: Ensure IDs are strings
disp_actions["taxnum"] = disp_actions["taxnum"].astype(str)
merged_with_personnel["Final_Officer_Tax_ID"] = merged_with_personnel["Final_Officer_Tax_ID"].astype(str)

# Step 2: Filter disp_actions to only taxnums that match Final_Officer_Tax_IDs
matched_disp_actions = disp_actions[disp_actions["taxnum"].isin(merged_with_personnel["Final_Officer_Tax_ID"])]

# Step 3: Check for duplicate taxnums in the matched subset
duplicate_taxnums = matched_disp_actions["taxnum"].value_counts()
duplicate_taxnums = duplicate_taxnums[duplicate_taxnums > 1]

# Step 4: Display results
print(f"Total unique matched taxnums: {matched_disp_actions['taxnum'].nunique()}")
print(f"Total matching rows in disp_actions: {len(matched_disp_actions)}")
print(f"Number of duplicate taxnums: {len(duplicate_taxnums)}")

if not duplicate_taxnums.empty:
    print("\n Duplicate matched taxnums (taxnum: count):")
    print(duplicate_taxnums)
else:
    print(" No duplicate matched taxnums found.")

```

```

Total unique matched taxnums: 16
Total matching rows in disp_actions: 72
Number of duplicate taxnums: 11

```

```

Duplicate matched taxnums (taxnum: count):
taxnum
948044    26
953225     8
935629     7
939265     5
938645     5
916969     3
941041     3
940354     3
955270     3
948300     2
938629     2
Name: count, dtype: int64

```

```

# Get unique values
unique_disp_taxnums = set(disp_actions["taxnum"].unique())
unique_personnel_ids = set(merged_with_personnel["Final_Officer_Tax_ID"].unique())

# Find the intersection (i.e., matches)
matched_taxnums = unique_disp_taxnums.intersection(unique_personnel_ids)

# Display the result
print(f"Unique taxnums in disp_actions: {len(unique_disp_taxnums):,}")

```

```
print(f"Unique Final_Officer_Tax_IDs in merged_with_personnel: {len(unique_personnel_ids):,}")  
print(f"Number of matching unique taxnums: {len(matched_taxnums):,}")
```

```
⇒ Unique taxnums in disp_actions: 237  
   Unique Final_Officer_Tax_IDs in merged_with_personnel: 2,428  
   Number of matching unique taxnums: 16
```

```
disp_actions[disp_actions["taxnum"] == "948044"]
```

	taxnum	command	charge	casenum	disposition	penalty	X8	
424	948044	042-042 PRECINCT	DID REPORT LATE FOR DUTY FOUR TIMES FOR A TOTA...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
425	948044	042-042 PRECINCT	WHILE OFF-DUTY, OPERATED HIS PERSONAL VEHICLE ...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
426	948044	042-042 PRECINCT	WHILE OFF-DUTY, OPERATED HIS PERSONAL VEHICLE ...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
427	948044	042-042 PRECINCT	WHILE OFF-DUTY, OPERATED HIS VEHICLE IN A RECK...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
428	948044	042-042 PRECINCT	DID FAIL TO NOTIFY THE DEPARTMENT THAT HE WAS ...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
429	948044	042-042 PRECINCT	DID OPERATE A MOTORCYCLE AND DID NOT HAVE A NE...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
430	948044	042-042 PRECINCT	WHILE ON-DUTY, MADE AN ATTEMPT TO ENGAGE ON-DU...	2016- 15245	GUILTY	Case: 2, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
431	948044	042-042 PRECINCT	WHILE ON-DUTY, FAILED TO NOTIFY THE PRECINCT D...	2016- 15245	PLEADED GUILTY	Case: 2, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
432	948044	042-042 PRECINCT	WHILE ON-DUTY, WAS DISMISSED FROM BRONX CRIMI...	2016- 15245	PLEADED GUILTY	Case: 2, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
433	948044	042-042 PRECINCT	WHILE ON-DUTY, FAILED TO ANSWER A SUPERVISOR, ...	2016- 15245	GUILTY	Case: 2, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
434	948044	579-MANHATTAN COURT SECTION	WHILE ASSIGNED TO MANHATTAN COURT SECTION ON A...	2017- 17346	GUILTY	Case: 3, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
435	948044	579-MANHATTAN COURT SECTION	WAS DISCOURTEOUS TO A PERSON KNOWN TO THE DEPA...	2017- 17346	GUILTY	Case: 3, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
436	948044	579-MANHATTAN COURT SECTION	FAIL AND NEGLECT TO REMOVE HIS DEPARTMENT IDEN...	2017- 17346	GUILTY	Case: 3, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
504	948044	042-042 PRECINCT	DID REPORT LATE FOR DUTY FOUR TIMES FOR A TOTA...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	
505	948044	042-042 PRECINCT	WHILE OFF-DUTY, OPERATED HIS PERSONAL VEHICLE ...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM	

506	948044	042-042 PRECINCT	WHILE OFF-DUTY, OPERATED HIS PERSONAL VEHICLE ...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM
507	948044	042-042 PRECINCT	WHILE OFF-DUTY, OPERATED HIS VEHICLE IN A RECK...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM
508	948044	042-042 PRECINCT	DID FAIL TO NOTIFY THE DEPARTMENT THAT HE WAS ...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM
509	948044	042-042 PRECINCT	DID OPERATE A MOTORCYCLE AND DID NOT HAVE A NE...	2015- 14565	PLEADED GUILTY	Case: 1, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM
510	948044	042-042 PRECINCT	WHILE ON-DUTY, MADE AN ATTEMPT TO ENGAGE ON-DU...	2016- 15245	GUILTY	Case: 2, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM
511	948044	042-042 PRECINCT	WHILE ON-DUTY, FAILED TO NOTIFY THE PRECINCT D	2016- 15245	PLEADED GUILTY	Case: 2, Penalty: DISMISSAL PROBATION(12 month...	04/08/16 12:00 AM

```
# Step 1: Ensure all taxnum are strings
disp_actions["taxnum"] = disp_actions["taxnum"].astype(str)
```

```
# Step 2: Group by taxnum and aggregate as lists
grouped_dispactions = disp_actions.groupby("taxnum").agg({
    "command": list,
    "charge": list,
    "casenum": list,
    "disposition": list,
    "penalty": list,
    "X8": list
}).reset_index()
```

```
# Merge into your large dataset
merged_with_disp = merged_with_personnel.merge(
    grouped_dispactions,
    how="left",
    left_on="Final_Officer_Tax_ID",
    right_on="taxnum"
).drop(columns=["taxnum"])
```

```
merged_with_disp.head()
```



	Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency	Con
0	190851430		PO	Soreca	953430	NYPD
1	196184708		.	Ennis	961743	NYPD
2	196241834		.	Wynno	961465	NYPD
3	196242292		.	Keniesky	964085	NYPD
4	196244840		.	Sinto	964778	NYPD

```
# Checking the columns
merged_with_disp.info()

23  Balance Due          4663 non-null  object
```



```

52 compliance_status      4663 non-null object
53 Charge #1: Code         3863 non-null object
54 Charge #1: Code Section 3859 non-null object
55 Charge #1: Code Description 3859 non-null object
56 Charge #1: Infraction Amount 4663 non-null object
57 Charge #2: Code         611 non-null object
58 Charge #2: Code Section 607 non-null object
59 Charge #2: Code Description 607 non-null object
60 Charge #2: Infraction Amount 1186 non-null object
61 Charge #3: Code         25 non-null object
62 Charge #3: Code Section 16 non-null object
63 Charge #3: Code Description 16 non-null object
64 Charge #3: Infraction Amount 36 non-null object
65 command_x              4513 non-null object
66 dept_recognitions       4513 non-null float64
67 rank                   4513 non-null object
68 shield                 4510 non-null float64
69 appt_date              4513 non-null object
70 total_arrests          4513 non-null float64
71 name                   4513 non-null object
72 race                   4513 non-null object
73 command_y             19 non-null object
74 charge                 19 non-null object
75 casenum                19 non-null object
76 disposition            19 non-null object
77 penalty                19 non-null object
78 X8                     19 non-null object
dtypes: float64(7), int64(2), object(70)
memory usage: 2.8+ MB

```

3. training_processed_clean.csv Merge

- Cannot merge because 9 million rows total (still coming up with resolution)

```

# Read in training_processed_clean.csv (too large to load)
training_processed_clean = pd.read_csv("/content/disp_actions5(1).csv")
training_processed_clean.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 588 entries, 0 to 587
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   taxnum      588 non-null    int64
1   command     587 non-null    object
2   charge      587 non-null    object
3   casenum     587 non-null    object
4   disposition 587 non-null    object
5   penalty     587 non-null    object
6   X8          588 non-null    object
dtypes: int64(1), object(6)
memory usage: 32.3+ KB

```

** 4. rs_processed_clean2.csv Merge **

- Rankings of Police Officers over time
- Multiple ranks assigned to one tax ID. How should we proceed? Either:
 - Include all rankings in one column; or
 - Match rank to ticket_number date (officer rank at time of citation)

```

# Read in rs_processed_clean2.csv Merge
rs_processed = pd.read_csv("/content/rs_processed_clean2.csv")
rs_processed.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53876 entries, 0 to 53875

```



```
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   taxnum       53876 non-null    int64
1   date         53868 non-null    object
2   shield_num   48987 non-null    float64
3   title        53868 non-null    object
dtypes: float64(1), int64(1), object(2)
memory usage: 1.6+ MB
```

```
# Quality Control: Checking for how may taxnum in personnel_file match with Final_Officer_Tax_ID in merged_df
rs_processed["taxnum"] = rs_processed["taxnum"].astype(str)
merged_with_disp["Final_Officer_Tax_ID"] = merged_with_disp["Final_Officer_Tax_ID"].astype(str)
```

```
# Create match mask
matches = merged_with_disp["Final_Officer_Tax_ID"].isin(rs_processed["taxnum"])
```

```
# Count matches
total_rows = len(merged_with_disp)
num_matches = matches.sum()
num_unmatched = total_rows - num_matches
match_percentage = round((num_matches / total_rows) * 100, 2)
# Display results
print(f"Total Rows in merged_with_personnel: {total_rows:,}")
print(f"Matched Tax IDs (with disp_actions): {num_matches:,}")
print(f"Unmatched Tax IDs: {num_unmatched:,}")
print(f"Match Percentage: {match_percentage}%")
```

```
↗ Total Rows in merged_with_personnel: 4,663
Matched Tax IDs (with disp_actions): 4,472
Unmatched Tax IDs: 191
Match Percentage: 95.9%
```

```
# Checking for duplicates: Officers with multiple disciplinary charges
```

```
# Step 1: Ensure IDs are strings
rs_processed["taxnum"] = rs_processed["taxnum"].astype(str)
merged_with_disp["Final_Officer_Tax_ID"] = merged_with_disp["Final_Officer_Tax_ID"].astype(str)
```

```
# Step 2: Filter rs_processed to only taxnums that match Final_Officer_Tax_IDs
matched_rs = rs_processed[rs_processed["taxnum"].isin(merged_with_disp["Final_Officer_Tax_ID"])]
```

```
# Step 3: Check for duplicate taxnums in the matched subset
duplicate_taxnums = matched_rs["taxnum"].value_counts()
duplicate_taxnums = duplicate_taxnums[duplicate_taxnums > 1]
```

```
# Step 4: Display results
print(f"Total unique matched taxnums: {matched_rs['taxnum'].nunique()}")
print(f"Total matching rows in rs_processed: {len(matched_rs)}")
print(f"Number of duplicate taxnums: {len(duplicate_taxnums)}")
```

```
if not duplicate_taxnums.empty:
    print("\n Duplicate matched taxnums (taxnum: count):")
    print(duplicate_taxnums)
else:
    print("✅ No duplicate matched taxnums found.")
```

```
↗ Total unique matched taxnums: 2288
Total matching rows in rs_processed: 2473
Number of duplicate taxnums: 164
```

```
Duplicate matched taxnums (taxnum: count):
taxnum
926603    6
945614    4
945059    4
944874    4
947524    3
```

```

    ..
954258    2
953819    2
954082    2
948887    2
956197    2
Name: count, Length: 164, dtype: int64

# Get unique values
unique_rs_taxnums = set(rs_processed["taxnum"].unique())
unique_merged_disp = set(merged_with_disp["Final_Officer_Tax_ID"].unique())

# Find the intersection (i.e., matches)
matched_taxnums = unique_rs_taxnums.intersection(unique_merged_disp)

# Display the result
print(f"Unique taxnums in rs: {len(unique_rs_taxnums):,}")
print(f"Unique Final_Officer_Tax_IDs in merged_with_disp: {len(unique_merged_disp):,}")
print(f"Number of matching unique taxnums: {len(matched_taxnums):,}")

```

```

⇒ Unique taxnums in rs: 34,219
   Unique Final_Officer_Tax_IDs in merged_with_disp: 2,428
   Number of matching unique taxnums: 2,288

```

```

# Calculate matching summary stats
total_disp_ids = merged_with_disp["Final_Officer_Tax_ID"].nunique()
matched_ids = matched_rs["taxnum"].nunique()
unmatched_ids = total_disp_ids - matched_ids
matched_percent = round((matched_ids / total_disp_ids) * 100, 2)

# Display results
print(f"\nMatched Tax ID: {matched_ids}")
print(f"Unmatched Tax ID: {unmatched_ids}")
print(f"Matched Percentage: {matched_percent}%")

```

```

⇒ Matched Tax ID: 2288
   Unmatched Tax ID: 140
   Matched Percentage: 94.23%

```

```

# Fix this!
# List of all duplicates
#duplicate_df.columns = ['taxnum', 'count']
#duplicate_df

# Count frequency of duplicate counts
#frequency_of_duplicates = duplicate_df["count"].value_counts().sort_index()

# Display results
#print(frequency_of_duplicates)

```

```

# Unique values in "title"
rs_processed["title"].unique()

```

```

⇒ array(['DETECTIVE', 'POLICE OFFICER', 'SERGEANT', 'LIEUTENANT', 'CAPTAIN',
        'CHIEF OF STRATEGIC INITIATIVES', 'DEPUTY CHIEF',
        'DEPUTY INSPECTOR', 'INSPECTOR', 'SGT DET SQUAD',
        'LT SPECIAL ASSIGN', 'LT DET COMMANDER', 'PO SPECIAL ASSIGN',
        'ASSISTANT CHIEF', nan, 'CHIEF OF DEPARTMENT', 'CHIEF OF TRAINING',
        'CHIEF OF LABOR REL', 'CHIEF OF TRANSIT',
        'DEPUTY COMMISSIONER - D', 'CHIEF OF DETECTIVES',
        'CHIEF OF INTELLIGENCE', 'CHIEF OF COLLABORATIVE POLICING',
        'CHIEF OF PATROL', 'TRAFFIC AGENT',

```

```
'CHIEF OF CRIME CNTRL STRATEGIES', 'CHIEF OF COMMUNITY AFFAIRS',
'CHIEF OF HOUSING', 'CHIEF OF PERSONNEL',
'CHIEF OF TRANSPORTATION', 'CHIEF OF SPECIAL OPERATIONS'],
dtype=object)
```

```
rs_processed[rs_processed["taxnum"] == "926603"]
```



	taxnum	date	shield_num	title
5617	926603	06/26/15	NaN	CAPTAIN
5618	926603	07/27/18	NaN	DEPUTY INSPECTOR
5619	926603	03/18/21	NaN	INSPECTOR
5620	926603	11/23/10	NaN	LIEUTENANT
5621	926603	09/29/00	10601.0	POLICE OFFICER
5622	926603	10/03/05	3880.0	SERGEANT




```
# Keeping the most recent rank and associated rank appt date
# Define rank order exactly as strings in your data
```

```
rank_order = {
    "INSPECTOR": 1,
    "DEPUTY INSPECTOR": 2,
    "CAPTAIN": 3,
    "LIEUTENANT": 4,
    "SERGEANT": 5,
    "DETECTIVE 3RD GRADE": 6,
    "DETECTIVE 2ND GRADE": 7,
    "DETECTIVE": 8,
    "POLICE OFFICER": 9
}
```

```
# create a separate 'rank_level' column
```

```
rs_processed["rank_level"] = rs_processed["title"].map(rank_order)
```

```
# Drop rows where rank is not recognized (i.e., title not in the dictionary)
rs_processed = rs_processed.dropna(subset=["rank_level"])
```

```
# Convert date to datetime (if needed)
```

```
rs_processed["date"] = pd.to_datetime(rs_processed["date"], errors="coerce")
```

```
# Sort: lowest rank_level = highest rank
```

```
rs_processed = rs_processed.sort_values(by=["taxnum", "rank_level", "date"])
```

```
# Keep only the highest rank per officer
```

```
rs_processed = rs_processed.drop_duplicates(subset=["taxnum"], keep="first")
```

```
# Cleanup helper column
```

```
rs_processed = rs_processed.drop(columns=["rank_level"])
```

```
rs_processed[rs_processed["taxnum"] == "926603"]
```



	taxnum	date	shield_num	title
5619	926603	2021-03-18	NaN	INSPECTOR



```
rs_processed.title.describe()
```



	title
count	34219
unique	7
top	POLICE OFFICER
freq	22034

dtype: object

```
# Step 1: Ensure all taxnum are strings
rs_processed["taxnum"] = rs_processed["taxnum"].astype(str)

grouped_rs = rs_processed.groupby("taxnum").agg({
    "title": list,
    "date": list,
    "shield_num": lambda x: int(x.dropna().iloc[0]) if x.dropna().any() else "."
}).reset_index()
```

```
grouped_rs[grouped_rs["taxnum"]== "926603"]
```



	taxnum	title	date	shield_num	
2523	926603	[INSPECTOR]	[2021-03-18 00:00:00]	.	



```
grouped_rs.title.describe()
```



	title
count	34219
unique	7
top	[POLICE OFFICER]
freq	22034

dtype: object

```
# Merge into your large dataset
merged_with_rs = merged_with_disp.merge(
    grouped_rs,
    how="left",
    left_on="Final_Officer_Tax_ID",
    right_on="taxnum"
).drop(columns=["taxnum"])
```

```
# Checking that all titles, dates, and shield_num go together.
merged_with_rs[merged_with_rs["Final_Officer_Tax_ID"] == "926603"]
```



Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency
---------------	-------------------------	------------------------	----------------------------	--------

2058	198293417	Andre	Brown	926603 NYPD
------	-----------	-------	-------	-------------

```
# Convert the 'title' column from a list to a string
# assuming you want to see unique titles assigned to officers (regardless of taxnum)
merged_with_rs['title'] = merged_with_rs['title'].apply(lambda x: ', '.join(x) if isinstance(x, list) else x)

# Now you can get the unique elements of the title column as strings
unique_titles = merged_with_rs['title'].unique()
print(unique_titles)
```

['POLICE OFFICER' nan 'SERGEANT' 'CAPTAIN' 'DETECTIVE' 'INSPECTOR' 'LIEUTENANT']

Start coding or [generate](#) with AI.

5. first_set_updates.csv Merge

```
# Read in first_set_updates.csv
# Check to match to merged_df
first_set_updates = pd.read_csv("/content/first_set_updated.csv")
first_set_updates.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34227 entries, 0 to 34226
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   row_num                34227 non-null  int64
1   taxnum                 34227 non-null  int64
2   name                   34227 non-null  object
3   first_name             34227 non-null  object
4   midd_initial           25500 non-null  object
5   lats_name              34227 non-null  object
6   rank                   34227 non-null  object
7   command                34227 non-null  object
8   shield                 31852 non-null  float64
9   appt_date              34227 non-null  object
10  total_arrests           34227 non-null  int64
11  dept_recognitions       34227 non-null  int64
12  ethnicity               33788 non-null  object
13  in_sample               34227 non-null  object
14  same_as_web_app         100 non-null    object
15  arrest count dif        29 non-null     float64
16  award count dif         5 non-null      float64
17  notes                   13 non-null     object
dtypes: float64(3), int64(4), object(11)
memory usage: 4.7+ MB
```

```
# Checking for duplicates in first_set_updates:
```

```
# Step 1: Ensure IDs are strings
first_set_updates["taxnum"] = first_set_updates["taxnum"].astype(str)
merged_with_rs["Final_Officer_Tax_ID"] = merged_with_rs["Final_Officer_Tax_ID"].astype(str)
```

```
# Step 2: Filter rs_processed to only taxnums that match Final_Officer_Tax_IDs
matched_first_set = first_set_updates[first_set_updates["taxnum"].isin(merged_with_rs["Final_Officer_Tax_ID"])]

# Step 3: Check for duplicate taxnums in the matched subset
duplicate_taxnums = matched_first_set["taxnum"].value_counts()
duplicate_taxnums = duplicate_taxnums[duplicate_taxnums > 1]

# Step 4: Display results
print(f"Total matched taxnums: {matched_first_set['taxnum'].nunique()}")
print(f"Total matching rows in first_set_updates: {len(matched_first_set)}")
print(f"Number of duplicate taxnums: {len(duplicate_taxnums)}")

if not duplicate_taxnums.empty:
    print("\n Duplicate matched taxnums (taxnum: count):")
    print(duplicate_taxnums)
else:
    print("No duplicate matched taxnums found.")
```

```
➦ Total matched taxnums: 2288
Total matching rows in first_set_updates: 2288
Number of duplicate taxnums: 0
No duplicate matched taxnums found.
```

```
# No duplicates, clean merge

# Quality Control: Check for tickets in clean_2021 that didn't find a match in the OATH dataset
total_tickets = len(clean_2021)

# Number of matched tickets by Ticket_Number
matched_count = merged_with_rs['Ticket_Number'].notnull().sum()

# Number of unmatched tickets by Ticket_Number
unmatched_count = merged_with_rs['Ticket_Number'].isnull().sum()

# Matched percentage
matched_pct = matched_count / total_tickets * 100

# Unmatched percentage
unmatched_pct = unmatched_count / total_tickets * 100

# Report
print(f"Total tickets: {total_tickets}")
print(f"Matched tickets: {matched_count} ({matched_pct:.2f}%)")
print(f"Unmatched tickets: {unmatched_count} ({unmatched_pct:.2f}%)")
```

```
# Retained all ticket_number from original clean_2021
```

```
➦ Total tickets: 4663
Matched tickets: 4663 (100.00%)
Unmatched tickets: 0 (0.00%)
```

```
# Merge
merged_with_firstset = merged_with_rs.merge(
    first_set_updates,
    how="left",
    left_on="Final_Officer_Tax_ID",
    right_on="taxnum"
).drop(columns=["taxnum"])
```

```
merged_with_firstset.head()
```



	Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency	Con
0	190851430		PO	Soreca	953430	NYPD
1	196184708	.		Ennis	961743	NYPD
2	196241834	.		Wynno	961465	NYPD
3	196242292	.		Keniesky	964085	NYPD
4	196244840	.		Sinto	964778	NYPD

```
# Check how many entries have multiple FS rank (appt) dates
multiple_appt_date_y_dates = merged_with_firstset["appt_date_y"].apply(lambda x: isinstance(x, list) and len(x) > 1)

# Count how many rows this applies to
print("Officers with multiple FS_dates:", multiple_appt_date_y_dates.sum())

# Optional: view them
merged_with_firstset[multiple_appt_date_y_dates][["Final_Officer_Tax_ID", "appt_date_y", "rank_y"]].head()
```



Officers with multiple FS_dates: 0

Final_Officer_Tax_ID	appt_date_y	rank_y

6. disciplinary_history_clean.csv Merge
- An officer has multiple charges. How to proceed:

```
# Read in disciplinary_history_clean.csv
disciplinary_history = pd.read_csv("/content/disciplinary_history_clean.csv")
disciplinary_history.info()
```



<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34523 entries, 0 to 34522
Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	taxnum	34523 non-null	int64

```

1  date                2434 non-null    object
2  charge_count        1785 non-null    float64
3  allegation_count     649 non-null    float64
dtypes: float64(2), int64(1), object(1)
memory usage: 1.1+ MB

```

```
# Checking for duplicates in disciplinary_history:
```

```
# Step 1: Ensure IDs are strings
```

```
disciplinary_history["taxnum"] = disciplinary_history["taxnum"].astype(str)
```

```
merged_with_firstset["Final_Officer_Tax_ID"] = merged_with_firstset["Final_Officer_Tax_ID"].astype(str)
```

```
# Step 2: Filter rs_processed to only taxnums that match Final_Officer_Tax_IDs
```

```
matched_dh = disciplinary_history[disciplinary_history["taxnum"].isin(merged_with_firstset["Final_Officer_Tax_ID"])]
```

```
# Step 3: Check for duplicate taxnums in the matched subset
```

```
duplicate_taxnums = matched_dh["taxnum"].value_counts()
```

```
duplicate_taxnums = duplicate_taxnums[duplicate_taxnums > 1]
```

```
# Step 4: Display results
```

```
print(f"Total matched taxnums: {matched_dh['taxnum'].nunique()}")
```

```
print(f"Total matching rows in disp_actions: {len(matched_dh)}")
```

```
print(f"Number of duplicate taxnums: {len(duplicate_taxnums)}")
```

```
if not duplicate_taxnums.empty:
```

```
    print("\n Duplicate matched taxnums (taxnum: count):")
```

```
    print(duplicate_taxnums)
```

```
else:
```

```
    print("✅ No duplicate matched taxnums found.")
```



```
Total matched taxnums: 2288
```

```
Total matching rows in disp_actions: 2304
```

```
Number of duplicate taxnums: 14
```

```
Duplicate matched taxnums (taxnum: count):
```

```
taxnum
```

```
953658    3
```

```
953225    3
```

```
967512    2
```

```
945059    2
```

```
956863    2
```

```
939265    2
```

```
941483    2
```

```
943441    2
```

```
948580    2
```

```
940420    2
```

```
916969    2
```

```
957101    2
```

```
949973    2
```

```
964470    2
```

```
Name: count, dtype: int64
```

```
# Quality Control: Checking for how may taxnum in personnel_file match with Final_Officer_Tax_ID in merged_df
```

```
disciplinary_history["taxnum"] = disciplinary_history["taxnum"].astype(str)
```

```
merged_with_firstset["Final_Officer_Tax_ID"] = merged_with_firstset["Final_Officer_Tax_ID"].astype(str)
```

```
# Create match mask
```

```
matches = merged_with_firstset["Final_Officer_Tax_ID"].isin(disciplinary_history["taxnum"])
```

```
# Count matches
```

```
total_rows = len(merged_with_firstset)
```

```
num_matches = matches.sum()
```

```
num_unmatched = total_rows - num_matches
```

```
match_percentage = round((num_matches / total_rows) * 100, 2)
```

```
# Display results
```

```
print(f"Total Rows: {total_rows},}")
```



```
print(f"Matched Tax IDs: {num_matches:,}")
print(f"Unmatched Tax IDs: {num_unmatched:,}")
print(f"Match Percentage: {match_percentage}%")
```

```
↗ Total Rows: 4,663
    Matched Tax IDs: 4,472
    Unmatched Tax IDs: 191
    Match Percentage: 95.9%
```

```
# Unique Final_Officer_Tax_IDs in merged_with_firstset
total_disp_ids = merged_with_firstset["Final_Officer_Tax_ID"].nunique()

# Unique matched IDs (those found in disciplinary_history["taxnum"])
matched_ids = merged_with_firstset[matches]["Final_Officer_Tax_ID"].nunique()

# Unmatched count
unmatched_ids = total_disp_ids - matched_ids

# Match percentage
matched_percent = round((matched_ids / total_disp_ids) * 100, 2)

# Display
print(f"\nMatched Tax ID: {matched_ids}")
print(f"Unmatched Tax ID: {unmatched_ids}")
print(f"Matched Percentage: {matched_percent}%")
```

```
↗ Matched Tax ID: 2288
    Unmatched Tax ID: 140
    Matched Percentage: 94.23%
```

```
# Ensure IDs are strings
rs_processed["taxnum"] = rs_processed["taxnum"].astype(str)
merged_with_firstset["Final_Officer_Tax_ID"] = merged_with_firstset["Final_Officer_Tax_ID"].astype(str)

# Get unique Final_Officer_Tax_IDs in main dataset
total_disp_ids = merged_with_firstset["Final_Officer_Tax_ID"].nunique()

# Get unique matched taxnums from rs_processed
matched_ids = rs_processed[rs_processed["taxnum"].isin(merged_with_firstset["Final_Officer_Tax_ID"])]["taxnum"]

# Calculate unmatched and match percentage
unmatched_ids = total_disp_ids - matched_ids
matched_percent = round((matched_ids / total_disp_ids) * 100, 2)

# Print results
print(f"Matched Tax ID: {matched_ids:,}")
print(f"Unmatched Tax ID: {unmatched_ids:,}")
print(f"Matched Percentage: {matched_percent}%")
```

```
↗ Matched Tax ID: 2,288
    Unmatched Tax ID: 140
    Matched Percentage: 94.23%
```

```
disciplinary_history[disciplinary_history["taxnum"] == "953658"]
```

```
↗
```

	taxnum	date	charge_count	allegation_count	
2061	953658	5/11/2020	5.0	NaN	
2062	953658	10/14/2020	1.0	NaN	
2063	953658	12/12/2021	NaN	1.0	

```
# Step 1: Ensure all taxnum are strings
disciplinary_history["taxnum"] = disciplinary_history["taxnum"].astype(str)

# Step 2: Group by taxnum and aggregate as lists
grouped_disphistory = disciplinary_history.groupby("taxnum").agg({
    "date": list,
    "charge_count": list,
    "allegation_count": list
}).reset_index()
```

```
grouped_disphistory[grouped_disphistory["taxnum"]== "953658"]
```



	taxnum	date	charge_count	allegation_count
18680	953658	[5/11/2020, 10/14/2020, 12/12/2021]	[5.0, 1.0, nan]	[nan, nan, 1.0]



```
# Step 1: Ensure both ID columns are strings
merged_with_firstset["Final_Officer_Tax_ID"] = merged_with_firstset["Final_Officer_Tax_ID"].astype(str)
disciplinary_history["taxnum"] = disciplinary_history["taxnum"].astype(str) # or your matching dataset

# Step 2: Get unique sets of IDs
unique_disp_ids = set(merged_with_firstset["Final_Officer_Tax_ID"].unique())
unique_match_ids = set(disciplinary_history["taxnum"].unique())

# Step 3: Compare sets
matched_ids = unique_disp_ids & unique_match_ids
unmatched_ids = unique_disp_ids - unique_match_ids

# Step 4: Summary
total_ids = len(unique_disp_ids)
matched_count = len(matched_ids)
unmatched_count = len(unmatched_ids)
matched_percent = round((matched_count / total_ids) * 100, 2)

# Step 5: Print results
print(f"Total Unique Final_Officer_Tax_IDs: {total_ids:,}")
print(f"Matched Tax IDs: {matched_count:,}")
print(f"Unmatched Tax IDs: {unmatched_count:,}")
print(f"Match Percentage: {matched_percent}%")
```



```
Total Unique Final_Officer_Tax_IDs: 2,428
Matched Tax IDs: 2,288
Unmatched Tax IDs: 140
Match Percentage: 94.23%
```

```
# Merge into your large dataset
merged_with_dh = merged_with_firstset.merge(
    grouped_disphistory,
    how="left",
    left_on="Final_Officer_Tax_ID",
    right_on="taxnum"
).drop(columns=["taxnum"])

merged_with_dh.head()
```



	Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency	Con
0	190851430		PO	Soreca	953430	NYPD
1	196184708	.		Ennis	961743	NYPD
2	196241834	.		Wynno	961465	NYPD
3	196242292	.		Keniesky	964085	NYPD
4	196244840	.		Sinto	964778	NYPD

7. charges_clean2.csv Merge

- charges_clean charges

```
# Read in charges_clean2.csv
charges_clean = pd.read_csv("/content/charges_clean2.csv")
charges_clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 37274 entries, 0 to 37273
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   taxnum          37274 non-null  int64
1   case_num        4681 non-null   object
2   charge          4680 non-null   object
3   case_value      4681 non-null   object
4   disposition     4681 non-null   object
dtypes: int64(1), object(4)
memory usage: 1.4+ MB
```

```
# Checking for duplicates in charges_clean:
```

```
# Step 1: Ensure IDs are strings
charges_clean["taxnum"] = charges_clean["taxnum"].astype(str)
merged_with_dh["Final_Officer_Tax_ID"] = merged_with_dh["Final_Officer_Tax_ID"].astype(str)
```

```
# Step 2: Filter rs_processed to only taxnums that match Final_Officer_Tax_IDs
```

```
matched_cc = charges_clean[charges_clean["taxnum"].isin(merged_with_dh["Final_Officer_Tax_ID"])]
```

```
# Step 3: Check for duplicate taxnums in the matched subset
```

```
duplicate_taxnums = matched_cc["taxnum"].value_counts()
```

```
duplicate_taxnums = duplicate_taxnums[duplicate_taxnums > 1]
```

```
# Step 4: Display results
```

```
print(f"Total matched taxnums: {matched_cc['taxnum'].nunique()}")
```

```
print(f"Total matching rows in rs_processed: {len(matched_cc)}")
```

```
print(f"Number of duplicate taxnums: {len(duplicate_taxnums)}")
```

```
if not duplicate_taxnums.empty:
```

```
    print("\n Duplicate matched taxnums (taxnum: count):")
```

```
    print(duplicate_taxnums)
```

```
else:
```

```
    print(" No duplicate matched taxnums found.")
```

```

960924      7
952322      6
953658      6
945059      6
935312      5
947721      5
933282      5
939265      5
941483      5
964470      5
938645      5
942214      5
961381      5
962062      4
953225      4
929928      4
945466      4
954718      4
916969      4
953289      4
962446      4
926281      3
952279      3
951501      3
956962      3
951228      3
943441      3
951057      3
958766      3
948606      3
945376      3
940354      3
930525      3
964828      3
952486      3
957101      3
958282      3
955270      3
941041      3
931364      3
948580      3
955814      3
948374      3
961279      2
954525      2

```

```
900950      2
938629      2
949361      2
Name: count, dtype: int64
```

```
# Step 1: Ensure both ID columns are strings
merged_with_dh["Final_Officer_Tax_ID"] = merged_with_dh["Final_Officer_Tax_ID"].astype(str)
charges_clean["taxnum"] = charges_clean["taxnum"].astype(str) # or your matching dataset

# Step 2: Get unique sets of IDs
unique_cc_ids = set(merged_with_dh["Final_Officer_Tax_ID"].unique())
unique_match_ids = set(charges_clean["taxnum"].unique())

# Step 3: Compare sets
matched_ids = unique_cc_ids & unique_match_ids
unmatched_ids = unique_cc_ids - unique_match_ids

# Step 4: Summary
total_ids = len(unique_cc_ids)
matched_count = len(matched_ids)
unmatched_count = len(unmatched_ids)
matched_percent = round((matched_count / total_ids) * 100, 2)

# Step 5: Print results
print(f"Total Unique Final_Officer_Tax_IDs: {total_ids:,}")
print(f"Matched Tax IDs: {matched_count:,}")
print(f"Unmatched Tax IDs: {unmatched_count:,}")
print(f"Match Percentage: {matched_percent}%")
```

➡ Total Unique Final_Officer_Tax_IDs: 2,428
Matched Tax IDs: 2,288
Unmatched Tax IDs: 140
Match Percentage: 94.23%

```
charges_clean[charges_clean["taxnum"] == "933518"]
```

	taxnum	case_num	charge	case_value	disposition	
14992	933518	2016-15046	WHILE ON-DUTY, UPON ARRIVING AT THE APARTMENT ...	Case: 1, Penalty: VACATION DAYS<=25 da...	GUILTY	
14993	933518	2016-15046	FAILED TO MAKE REQUIRED ENTRIES IN HIS ACTIVIT...	Case: 1, Penalty: VACATION DAYS<=25 da...	GUILTY	
14994	933518	2016-15046	FAILED TO PROPERLY SAFEGUARD UNITED STATES CUR...	Case: 1, Penalty: VACATION DAYS<=25 da...	GUILTY	
14995	933518	2016-15046	WHILE ON-DUTY, FAILED TO PREPARE A COLLISION E...	Case: 1, Penalty: VACATION DAYS<=25 da...	GUILTY	
14996	933518	2016-15046	WHILE ON-DUTY, FAILED TO PREPARE A COLLISION I...	Case: 1, Penalty: VACATION DAYS<=25 da...	GUILTY	
14997	933518	2016-15046	WHILE ON-DUTY, FAILED TO NOTIFY THE PATROL SUP...	Case: 1, Penalty: VACATION DAYS<=25 da...	GUILTY	
14998	933518	2016-15046	WHILE ON-DUTY, FAILED TO MAKE REQUIRED ENTRIES...	Case: 1, Penalty: VACATION DAYS<=25 da...	GUILTY	

```
# Step 1: Ensure all taxnum are strings
charges_clean["taxnum"] = charges_clean["taxnum"].astype(str)

# Step 2: Group by taxnum and aggregate as lists
grouped_cc = charges_clean.groupby("taxnum").agg({
    "case_num": list,
    "charge": list,
    "case_value": list,
    "disposition": list
})
```

```
}).reset_index()
```

```
grouped_cc[grouped_cc["taxnum"] == "933518"]
```

	taxnum	case_num	charge	case_value	disposition
5152	933518	[2016-15046, 2016-15046, 2016-15046, 2016-1504...	[WHILE ON-DUTY, UPON ARRIVING AT THE APARTMENT...	[Case: 1, Penalty: VACATION DAYS <i>(25 d...	[GUILTY, GUILTY, GUILTY, GUILTY, GUILTY, GUILT...

```
# Merge into your large dataset
merged_with_cc = merged_with_dh.merge(
    grouped_cc,
    how="left",
    left_on="Final_Officer_Tax_ID",
    right_on="taxnum"
).drop(columns=["taxnum"])
```

```
merged_with_cc[merged_with_cc["Final_Officer_Tax_ID"] == "933518"]
```

	Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency
668	203005972	.	Herrmann	933518	NYPD

7. arrest_all.csv Merge

- Group award by type and count

```
# Read in arrest_all.csv
arrest_all = pd.read_csv("/content/arrest_all.csv")
arrest_all.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 101648 entries, 0 to 101647
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   taxnum      101648 non-null  int64
1   arrest_count 100444 non-null  float64
2   arrest_type  100444 non-null  object
dtypes: float64(1), int64(1), object(1)
memory usage: 2.3+ MB
```

```
# Quality Control:
```

```
# Step 1: Ensure IDs are strings
arrest_all["taxnum"] = arrest_all["taxnum"].astype(str)
merged_with_cc["Final_Officer_Tax_ID"] = merged_with_cc["Final_Officer_Tax_ID"].astype(str)
```

```
# Step 2: Filter rs_processed to only taxnums that match Final_Officer_Tax_IDs
matched_aa = arrest_all[arrest_all["taxnum"].isin(merged_with_cc["Final_Officer_Tax_ID"])]
```

```
# Step 3: Check for duplicate taxnums in the matched subset
duplicate_taxnums = matched_aa["taxnum"].value_counts()
duplicate_taxnums = duplicate_taxnums[duplicate_taxnums > 1]
```

```
# Step 4: Display results
print(f"Total matched taxnums: {matched_aa['taxnum'].nunique()}")
print(f"Total matching rows in arrest_all: {len(matched_aa)}")
print(f"Number of duplicate taxnums: {len(duplicate_taxnums)}")
```

```
if not duplicate_taxnums.empty:
    print("\n Duplicate matched taxnums (taxnum: count):")
    print(duplicate_taxnums)
else:
    print("No duplicate matched taxnums found.")
```

```
➦ Total matched taxnums: 2288
Total matching rows in arrest_all: 6144
Number of duplicate taxnums: 2243
```

```
Duplicate matched taxnums (taxnum: count):
taxnum
938755    5
935100    5
923667    5
959710    5
951019    5
..
970950    2
970335    2
971279    2
969437    2
959398    2
Name: count, Length: 2243, dtype: int64
```

```
# Step 1: Ensure both ID columns are strings
merged_with_cc["Final_Officer_Tax_ID"] = merged_with_cc["Final_Officer_Tax_ID"].astype(str)
arrest_all["taxnum"] = arrest_all["taxnum"].astype(str)
```

```
# Step 2: Get unique sets of IDs
unique_aa_ids = set(merged_with_cc["Final_Officer_Tax_ID"].unique())
unique_match_ids = set(arrest_all["taxnum"].unique())
```

```
# Step 3: Compare sets
matched_ids = unique_aa_ids & unique_match_ids
unmatched_ids = unique_aa_ids - unique_match_ids
```

```
# Step 4: Summary
total_ids = len(unique_aa_ids)
matched_count = len(matched_ids)
unmatched_count = len(unmatched_ids)
matched_percent = round((matched_count / total_ids) * 100, 2)
```

```
# Step 5: Print results
print(f"Total Unique Final_Officer_Tax_IDs: {total_ids:,}")
print(f"Matched Tax IDs: {matched_count:,}")
print(f"Unmatched Tax IDs: {unmatched_count:,}")
print(f"Match Percentage: {matched_percent}%")
```

```
➦ Total Unique Final_Officer_Tax_IDs: 2,428
Matched Tax IDs: 2,288
Unmatched Tax IDs: 140
Match Percentage: 94.23%
```

```
arrest_all[arrest_all["taxnum"] == "938755"]
```



	taxnum	arrest_count	arrest_type
46424	938755	2.0	INFRACTION
46425	938755	229.0	FELONY
46426	938755	294.0	MISDEMEANOR
46427	938755	1.0	OTHER
46428	938755	9.0	VIOLATION

```
arrest_all.arrest_type.unique()
```

```
array(['INFRACTION', 'FELONY', 'MISDEMEANOR', 'OTHER', 'VIOLATION', nan],
      dtype=object)
```

```
# Step 1: Ensure all taxnum are strings
#arrest_all["taxnum"] = arrest_all["taxnum"].astype(str)
```

```
# Step 2: Group by taxnum and aggregate as lists
#grouped_aa = arrest_all.groupby("taxnum").agg({
#    "arrest_count": list,
#    "arrest_type": list
# }).reset_index()
```

```
#grouped_aa[grouped_aa["taxnum"] == "938755"]
```

```
# Step 1: Ensure taxnum is a string
arrest_all["taxnum"] = arrest_all["taxnum"].astype(str)
```

```
# Step 2: Pivot arrest types into columns
pivot_aa = arrest_all.pivot_table(
    index="taxnum",
    columns="arrest_type",
    values="arrest_count",
    aggfunc="sum"
).reset_index()
```

```
# Step 3: Fill NaNs with 0s
pivot_aa = pivot_aa.fillna(0)
```

```
# Step 4: Rename columns to match desired format
pivot_aa.columns.name = None
pivot_aa = pivot_aa.rename(columns={
    "FELONY": "aa_felony_arrest_count",
    "INFRACTION": "aa_infraction_arrest_count",
    "MISDEMEANOR": "aa_misdemeanor_arrest_count",
    "VIOLATION": "aa_violation_arrest_count",
    "OTHER": "aa_other_arrest_count"
})
```

```
# Step 5: Merge with main dataset
merged_with_aa = merged_with_cc.merge(
    pivot_aa,
    how="left",
    left_on="Final_Officer_Tax_ID",
    right_on="taxnum"
).drop(columns=["taxnum"])
```

```
# Step 6 (Optional): Fill any post-merge NaNs as 0
arrest_cols = [
```



```

    "aa_felony_arrest_count", "aa_infraction_arrest_count", "aa_misdemeanor_arrest_count",
    "aa_violation_arrest_count", "aa_other_arrest_count"
]
merged_with_aa[arrest_cols] = merged_with_aa[arrest_cols].fillna(0)

merged_with_aa[
    [
        "Final_Officer_Tax_ID",
        "aa_felony_arrest_count",
        "aa_infraction_arrest_count",
        "aa_misdemeanor_arrest_count",
        "aa_violation_arrest_count",
        "aa_other_arrest_count"
    ]
].head(10) # You can change 10 to any number of rows you want to preview

```



	Final_Officer_Tax_ID	aa_felony_arrest_count	aa_infraction_arrest_count	aa_misdemeanor_arrest_count	aa
0	953430	25.0	0.0	92.0	
1	961743	24.0	0.0	39.0	
2	961465	21.0	2.0	39.0	
3	964085	12.0	0.0	23.0	
4	964778	40.0	0.0	40.0	
5	963591	19.0	0.0	38.0	
6	963656	15.0	0.0	33.0	
7	952904	35.0	1.0	203.0	
8	966902	16.0	0.0	35.0	
9	966375	0.0	0.0	0.0	

```
merged_with_aa
```



Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency
0	190851430	PO	Soreca	953430 NYPD
1	196184708	.	Ennis	961743 NYPD
2	196241834	.	Wynno	961465 NYPD
3	196242292	.	Keniesky	964085 NYPD
4	196244840	.	Sinto	964778 NYPD
...	...	...	...	...
4658	211740870	.	C	952535 NYPD
4659	211741072	.	Rodriguez	953330 NYPD
4660	211744895	.	Blackman	948461 NYPD
4661	211744904	.	Blackman	948461 NYPD
4662	211746691	.	Singh	954342 NYPD

4663 rows x 111 columns

8. ar_processed_clean.csv Merge

- 15 types of awards
- Do we need the date corresponding to award given? or do we care more about type of award given?
- Proposition: Have 15 columns for each award and count of awards for each officer

```
# Read in ar_processed_clean.csv
award_processed = pd.read_csv("/content/ar_processed_clean.csv")
award_processed.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 143904 entries, 0 to 143903
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   taxnum  143904 non-null   int64
 1   date    128490 non-null   object
 2   award   128670 non-null   object
dtypes: int64(1), object(2)
memory usage: 3.3+ MB
```

```
# Checking for duplicates in ar_processed_clean:
```

```
# Step 1: Ensure IDs are strings
award_processed["taxnum"] = award_processed["taxnum"].astype(str)
merged_with_aa["Final_Officer_Tax_ID"] = merged_with_aa["Final_Officer_Tax_ID"].astype(str)

# Step 2: Filter rs_processed to only taxnums that match Final_Officer_Tax_IDs
matched_award = award_processed[award_processed["taxnum"].isin(merged_with_aa["Final_Officer_Tax_ID"])]

# Step 3: Check for duplicate taxnums in the matched subset
duplicate_taxnums = matched_award["taxnum"].value_counts()
duplicate_taxnums = duplicate_taxnums[duplicate_taxnums > 1]

# Step 4: Display results
print(f"Total matched taxnums: {matched_award['taxnum'].nunique()}")
print(f"Total matching rows in rs_processed: {len(matched_award)}")
print(f"Number of duplicate taxnums: {len(duplicate_taxnums)}")

if not duplicate_taxnums.empty:
    print("\n Duplicate matched taxnums (taxnum: count):")
    print(duplicate_taxnums)
else:
    print("No duplicate matched taxnums found.")
```

```
Total matched taxnums: 2288
Total matching rows in rs_processed: 4748
Number of duplicate taxnums: 596

Duplicate matched taxnums (taxnum: count):
taxnum
954525    48
957529    32
948940    31
955270    29
961483    28
..
957372     2
959688     2
960728     2
962604     2
965996     2
Name: count, Length: 596, dtype: int64
```

```
award_processed[award_processed["taxnum"] == "954525"]
```



	taxnum	date	award
8807	954525	12/20/2014	COMMENDATION
8808	954525	8/22/2015	EXCELLENT POLICE DUTY
8809	954525	9/22/2015	EXCELLENT POLICE DUTY
8810	954525	12/10/2015	EXCELLENT POLICE DUTY
8811	954525	1/22/2016	EXCELLENT POLICE DUTY
8812	954525	2/1/2016	MERITORIOUS POLICE DUTY
8813	954525	4/1/2016	EXCELLENT POLICE DUTY
8814	954525	4/13/2016	MERITORIOUS POLICE DUTY
8815	954525	4/22/2016	EXCELLENT POLICE DUTY
8816	954525	6/6/2016	EXCELLENT POLICE DUTY
8817	954525	7/22/2016	MERITORIOUS POLICE DUTY
8818	954525	8/6/2016	EXCELLENT POLICE DUTY
8819	954525	8/14/2016	EXCELLENT POLICE DUTY
8820	954525	9/18/2016	EXCELLENT POLICE DUTY
8821	954525	10/1/2016	EXCELLENT POLICE DUTY
8822	954525	10/26/2016	EXCELLENT POLICE DUTY
8823	954525	11/4/2016	EXCELLENT POLICE DUTY
8824	954525	11/29/2016	EXCELLENT POLICE DUTY
8825	954525	12/17/2016	EXCELLENT POLICE DUTY
8826	954525	1/7/2017	EXCELLENT POLICE DUTY
8827	954525	2/26/2017	EXCELLENT POLICE DUTY
8828	954525	1/18/2018	EXCELLENT POLICE DUTY
8829	954525	4/26/2018	EXCELLENT POLICE DUTY
8830	954525	5/12/2018	MERITORIOUS POLICE DUTY
8831	954525	10/30/2018	EXCELLENT POLICE DUTY
8832	954525	10/31/2018	MERITORIOUS POLICE DUTY
8833	954525	1/10/2019	EXCELLENT POLICE DUTY
8834	954525	4/4/2019	MERITORIOUS POLICE DUTY
8835	954525	5/24/2019	EXCELLENT POLICE DUTY
8836	954525	7/11/2019	EXCELLENT POLICE DUTY
8837	954525	11/29/2019	EXCELLENT POLICE DUTY
8838	954525	2/27/2020	EXCELLENT POLICE DUTY
8839	954525	4/9/2020	EXCELLENT POLICE DUTY
8840	954525	5/27/2020	MERITORIOUS POLICE DUTY
8841	954525	5/30/2020	EXCELLENT POLICE DUTY
8842	954525	6/2/2020	EXCELLENT POLICE DUTY
8843	954525	6/14/2020	EXCELLENT POLICE DUTY
8844	954525	8/18/2020	MERITORIOUS POLICE DUTY
8845	954525	9/1/2020	EXCELLENT POLICE DUTY
8846	954525	9/6/2020	EXCELLENT POLICE DUTY



```

8847  954525  10/20/2020  EXCELLENT POLICE DUTY
8848  954525  11/6/2020  MERITORIOUS POLICE DUTY
8849  954525  12/10/2020  MERITORIOUS POLICE DUTY
8850  954525  12/18/2020  MERITORIOUS POLICE DUTY
8851  954525  2/2/2021    EXCELLENT POLICE DUTY
8852  954525  2/10/2021  MERITORIOUS POLICE DUTY
8853  954525  2/17/2021  MERITORIOUS POLICE DUTY
8854  954525  2/23/2021  MERITORIOUS POLICE DUTY

```

```

# View all unique award types in the dataset
# 15 different award types
award_type = award_processed["award"].unique()
award_type
#len(award_type)

```

```

array(['EXCELLENT POLICE DUTY', 'MERITORIOUS POLICE DUTY', nan,
      'HONORABLE MENTION', 'MEDAL FOR VALOR', 'COMMENDATION',
      'PURPLE SHIELD MEDAL', 'MERITORIOUS POLICE DUTY-INTEGRITY',
      'COMMENDATION - COMMUNITY SERVICE', 'POLICE COMBAT CROSS',
      'EXCEPTIONAL MERIT', 'COMMENDATION-INTEGRITY', 'UNIT CITATION',
      'MEDAL FOR MERIT', 'MEDAL OF HONOR'], dtype=object)

```

```

# Filter for the officer
officer_awards = award_processed[award_processed["taxnum"] == "954525"]

```

```

# Count occurrences of each award type
award_counts = officer_awards["award"].value_counts()

award_counts

```

```

count
award
EXCELLENT POLICE DUTY    32
MERITORIOUS POLICE DUTY    15
COMMENDATION              1

```

dtype: int64

```

# Step 1: Ensure all taxnum are strings
award_processed["taxnum"] = award_processed["taxnum"].astype(str)

```

```

# Step 2: Group by taxnum and aggregate as lists
grouped_award = award_processed.groupby("taxnum").agg({
    "date": list,
    "award": list
}).reset_index()

```

```
grouped_award[grouped_award["taxnum"] == "954525"]
```

```

taxnum    date    award
19334  954525  [12/20/2014, 8/22/2015, 9/22/2015, 12/10/2015,...] [COMMENDATION, EXCELLENT POLICE DUTY, EXCELLEN...

```

```
# Merge into your large dataset
merged_with_award = merged_with_aa.merge(
    grouped_award,
    how="left",
    left_on="Final_Officer_Tax_ID",
    right_on="taxnum"
).drop(columns=["taxnum"])
```

✓ 9. allegation_detail_clean.csv Merge

```
# Read in allegation_detail_clean.csv
allegation_det = pd.read_csv("/content/allegation_detail_clean.csv")
allegation_det.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34608 entries, 0 to 34607
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   taxnum                34608 non-null  int64
1   case_num              1003 non-null   object
2   allegation            1003 non-null   object
3   case_value            1003 non-null   object
4   recommendation        1003 non-null   object
dtypes: int64(1), object(4)
memory usage: 1.3+ MB
```

```
# Step 1: Ensure IDs are strings
allegation_det["taxnum"] = allegation_det["taxnum"].astype(str)
merged_with_award["Final_Officer_Tax_ID"] = merged_with_award["Final_Officer_Tax_ID"].astype(str)
```

```
# Step 2: Filter allegation_det to only taxnums that match Final_Officer_Tax_IDs
matched_alleg = allegation_det[allegation_det["taxnum"].isin(merged_with_award["Final_Officer_Tax_ID"])]
```

```
# Step 3: Check for duplicate taxnums in the matched subset
duplicate_taxnums = matched_alleg["taxnum"].value_counts()
duplicate_taxnums = duplicate_taxnums[duplicate_taxnums > 1]
```

```
# Step 4: Display results
print(f"Total matched taxnums: {matched_alleg['taxnum'].nunique()}")
print(f"Total matching rows in rs_processed: {len(matched_alleg)}")
print(f"Number of duplicate taxnums: {len(duplicate_taxnums)}")
```

```
if not duplicate_taxnums.empty:
    print("\n Duplicate matched taxnums (taxnum: count):")
    print(duplicate_taxnums)
else:
    print("No duplicate matched taxnums found.")
```

```
Total matched taxnums: 2288
Total matching rows in rs_processed: 2312
Number of duplicate taxnums: 15

Duplicate matched taxnums (taxnum: count):
taxnum
967512    7
958234    4
965787    4
957734    2
956863    2
966238    2
965840    2
968548    2
965869    2
960090    2
964956    2
965788    2
```

```

960880    2
957583    2
965125    2
Name: count, dtype: int64

```

```
allegation_det[allegation_det["taxnum"] == "967512"]
```

	taxnum	case_num	allegation	case_value	recommendation	
10733	967512	2022-25429	01.Failed to activate their body-worn camera.	Case: 1, Penalty: C - COMMAND DISCIPLINE, VACA...	C-CD	
10734	967512	2022-25429	02.Failed to make required entries in Departme...	Case: 1, Penalty: C - COMMAND DISCIPLINE, VACA...	C-CD	
10735	967512	2022-25429	03.Failed to notify the radio dispatcher about...	Case: 1, Penalty: C - COMMAND DISCIPLINE, VACA...	C-CD	
10736	967512	2022-25429	04.Improper vehicle pursuit.	Case: 1, Penalty: C - COMMAND DISCIPLINE, VACA...	C-CD	
10737	967512	2022-25981	01.Violated patrol rules.	Case: 2	A-CD	
10738	967512	2022-25982	01.Failed to activate their body worn camera.	Case: 3	OTHER	
10739	967512	2022-25982	02.Violated patrol rules.	Case: 3	OTHER	

```

# Step 1: Ensure all taxnum are strings
allegation_det["taxnum"] = allegation_det["taxnum"].astype(str)

```

```

# Step 2: Group by taxnum and aggregate as lists
grouped_alleg = allegation_det.groupby("taxnum").agg({
    "case_num": list,
    "allegation": list,
    "case_value": list,
    "recommendation": list
}).reset_index()

```

```
grouped_alleg[grouped_alleg["taxnum"] == "967512"]
```

	taxnum	case_num	allegation	case_value	recommendation	
28846	967512	[2022-25429, 2022-25429, 2022-25429, 2022-2542...	[01.Failed to activate their body-worn camera....	[Case: 1, Penalty: C - COMMAND DISCIPLINE, VAC...	[C-CD, C-CD, C-CD, C-CD, A-CD, OTHER, OTHER]	

```

# Merge into your large dataset
merged_with_alleg = merged_with_award.merge(
    grouped_alleg,
    how="left",
    left_on="Final_Officer_Tax_ID",
    right_on="taxnum"
).drop(columns=["taxnum"])

```

```
merged_with_alleg.head()
```



	Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency	Con
0	190851430		PO	Soreca	953430	NYPD
1	196184708		.	Ennis	961743	NYPD
2	196241834		.	Wynno	961465	NYPD
3	196242292		.	Keniesky	964085	NYPD
4	196244840		.	Sinto	964778	NYPD

```
column_names_list = merged_with_alleg.columns.tolist()
column_names_list
```




```

command',
'shield_y',
'appt_date_y',
'total_arrests_y',
'dept_recognitions_y',
'ethnicity',
'in_sample',
'same_as_web_app',
'arrest count dif',
'award count dif',
'notes',
'date_y',
'charge_count',
'allegation_count',
'case_num_x',
'charge_y',
'case_value_x',
'disposition_y',
'aa_felony_arrest_count',
'aa_infraction_arrest_count',
'aa_misdemeanor_arrest_count',
'aa_other_arrest_count',
'aa_violation_arrest_count',
'date',
'award',
'case_num_y',
'allegation',
'case_value_y',
'recommendation']

```

✓ Renaming Variables for Clarity

- personnel_file variables : PF

```

# Rename personnel_file variables
draft_demographic_merge = merged_with_alleg.rename(columns={
    'command_x': 'PF_command',
    'rank_x': 'PF_rank',
    'dept_recognitions_x': 'PF_dept_recognitions',
    'dept_awards_x': 'PF_dept_awards',
    'rank_x': 'PF_rank',
    'shield_x': 'PF_shield',
    'appt_date_x': 'PF_appt_date',
    'total_arrests_x': 'PF_total_arrests',
    'name_x': 'PF_lname_fname',
    'race': 'PF_race'
})

```

```
draft_demographic_merge.head()
```



	Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency	Con
0	190851430		PO	Soreca	953430	NYPD
1	196184708		.	Ennis	961743	NYPD
2	196241834		.	Wynno	961465	NYPD
3	196242292		.	Keniesky	964085	NYPD
4	196244840		.	Sinto	964778	NYPD

```
column_names_list = draft_demographic_merge.columns.tolist()
column_names_list
```



```

    'command_y',
    'shield_y',
    'appt_date_y',
    'total_arrests_y',
    'dept_recognitions_y',
    'ethnicity',
    'in_sample',
    'same_as_web_app',
    'arrest count dif',
    'award count dif',
    'notes',
    'date_y',
    'charge_count',
    'allegation_count',
    'case_num_x',
    'charge_y',
    'case_value_x',
    'disposition_y',
    'aa_felony_arrest_count',
    'aa_infraction_arrest_count',
    'aa_misdemeanor_arrest_count',
    'aa_other_arrest_count',
    'aa_violation_arrest_count',
    'date',
    'award',
    'case_num_y',
    'allegation',
    'case_value_y',
    'recommendation']

```

```
# Rename disp_action variables
```

```

draft_demographic_merge = draft_demographic_merge.rename(columns={
    'command_y': 'DA_command',
    'charge_x': 'DA_charge',
    'casenum': 'DA_casenum',
    'disposition_x': 'DA_disposition',
    'penalty': 'DA_penalty',
    'X8': 'DA_X8'
})

```

```

column_names_list = draft_demographic_merge.columns.tolist()
column_names_list

```



```
rank_y ,
'command',
'shield_y',
'appt_date_y',
'total_arrests_y',
'dept_recognitions_y',
'ethnicity',
'in_sample',
'same_as_web_app',
'arrest count dif',
'award count dif',
'notes',
'date_y',
'charge_count',
'allegation_count',
'case_num_x',
'charge_y',
'case_value_x',
'disposition_y',
'aa_felony_arrest_count',
'aa_infraction_arrest_count',
'aa_misdemeanor_arrest_count',
'aa_other_arrest_count',
'aa_violation_arrest_count',
'date',
'award',
'case_num_y',
'allegation',
'case_value_y',
'recommendation']
```

```
draft_demographic_merge.head()
```



	Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency	Con
0	190851430		PO	Soreca	953430	NYPD
1	196184708	.		Ennis	961743	NYPD
2	196241834	.		Wynno	961465	NYPD
3	196242292	.		Keniesky	964085	NYPD
4	196244840	.		Sinto	964778	NYPD

```
# Rename rs_processed variables
draft_demographic_merge = draft_demographic_merge.rename(columns={
    'title': 'RS_rank',
    'date_x': 'RS_date',
    'shield_num': 'RS_shield'
})
```

```
column_names_list = draft_demographic_merge.columns.tolist()
column_names_list
```

```
↳ ['Charge #2: Code Description',
   'Charge #2: Infraction Amount',
   'Charge #3: Code',
   'Charge #3: Code Section',
   'Charge #3: Code Description',
   'Charge #3: Infraction Amount',
   'PF_command',
   'PF_dept_recognitions',
   'PF_rank',
   'PF_shield',
   'PF_appt_date',
   'PF_total_arrests',
   'PF_lname_fname',
   'PF_race',
   'DA_command',
   'DA_charge',
   'DA_casenum',
   'DA_disposition',
   'DA_penalty',
   'DA_X8',
   'RS_rank',
   'RS_date',
   'RS_shield',
   'row_num',
   'name_y',
   'first_name',
   'midd_initial',
   'lats_name',
   'rank_y',
   'command',
   'shield_y',
   'appt_date_y',
   'total_arrests_y',
   'dept_recognitions_y',
   'ethnicity',
   'in_sample',
   'same_as_web_app',
   'arrest count dif',
   'award count dif',
   'notes',
   'date_y',
   'charge_count',
   'allegation_count',
   'case_num_x',
   'charge_y',
   'case_value_x',
   'disposition_y',
   'aa_felony_arrest_count',
   'aa_infraction_arrest_count',
   'aa_misdemeanor_arrest_count',
   'aa_other_arrest_count',
   'aa_violation_arrest_count',
   'date',
   'award',
   'case_num_y',
   'allegation',
   'case_value_y',
   'recommendation']
```

```
draft_demographic_merge.head()
```



	Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency	Con
0	190851430		PO	Soreca	953430	NYPD
1	196184708	.		Ennis	961743	NYPD
2	196241834	.		Wynno	961465	NYPD
3	196242292	.		Keniesky	964085	NYPD
4	196244840	.		Sinto	964778	NYPD

```
# Drop row number (unnecessary column counting rows)
draft_demographic_merge = draft_demographic_merge.drop(columns=['row_num'])
```

```
# Rename first_set_updated variables
draft_demographic_merge = draft_demographic_merge.rename(columns={
    'name_y': 'FS_lname_fname',
    'first_name': 'FS_first_name',
    'midd_initial': 'FS_middle_name',
    'lats_name': 'FS_last_name',
    'rank_y': 'FS_rank',
    'command': 'FS_command',
    'shield_y': 'FS_shield',
    'appt_date_y': 'FS_appt_date',
    'total_arrests_y': 'FS_total_arrests',
    'dept_recognitions_y': 'FS_dept_recognitions',
    'ethnicity': 'FS_ethnicity',
    'in_sample': 'FS_in_sample',
    'same_as_web_app': 'FS_same_as_web_app',
    'arrest count dif': 'FS_arrest_count_dif',
    'award count dif': 'FS_award_count_dif',
    'notes': 'FS_notes'
})
```

```
column_names_list = draft_demographic_merge.columns.tolist()
column_names_list
```



```

Charge #2: Infraction Amount',
'Charge #3: Code',
'Charge #3: Code Section',
'Charge #3: Code Description',
'Charge #3: Infraction Amount',
'PF_command',
'PF_dept_recognitions',
'PF_rank',
'PF_shield',
'PF_appt_date',
'PF_total_arrests',
'PF_lname_fname',
'PF_race',
'DA_command',
'DA_charge',
'DA_casenum',
'DA_disposition',
'DA_penalty',
'DA_X8',
'RS_rank',
'RS_date',
'RS_shield',
'FS_lname_fname',
'FS_first_name',
'FS_middle_name',
'FS_last_name',
'FS_rank',
'FS_command',
'FS_shield',
'FS_appt_date',
'FS_total_arrests',
'FS_dept_recognitions',
'FS_ethnicity',
'FS_in_sample',
'FS_same_as_web_app',
'FS_arrest_count_dif',
'FS_award_count_dif',
'FS_notes',
'date_y',
'charge_count',
'allegation_count',
'case_num_x',
'charge_y',
'case_value_x',
'disposition_y',
'aa_felony_arrest_count',
'aa_infraction_arrest_count',
'aa_misdemeanor_arrest_count',
'aa_other_arrest_count',
'aa_violation_arrest_count',
'date',
'award',
'case_num_y',
'allegation',
'case_value_y',
'recommendation']

# Rename disciplinary_history variables
draft_demographic_merge = draft_demographic_merge.rename(columns={
    'date_y': 'DH_date',
    'charge_count': 'DH_charge_count',
    'allegation_count': 'DH_allegation_count'
})

column_names_list = draft_demographic_merge.columns.tolist()
column_names_list

```



```

    'charge #3: code description',
    'Charge #3: Infraction Amount',
    'PF_command',
    'PF_dept_recognitions',
    'PF_rank',
    'PF_shield',
    'PF_appt_date',
    'PF_total_arrests',
    'PF_lname_fname',
    'PF_race',
    'DA_command',
    'DA_charge',
    'DA_casenum',
    'DA_disposition',
    'DA_penalty',
    'DA_X8',
    'RS_rank',
    'RS_date',
    'RS_shield',
    'FS_lname_fname',
    'FS_first_name',
    'FS_middle_name',
    'FS_last_name',
    'FS_rank',
    'FS_command',
    'FS_shield',
    'FS_appt_date',
    'FS_total_arrests',
    'FS_dept_recognitions',
    'FS_ethnicity',
    'FS_in_sample',
    'FS_same_as_web_app',
    'FS_arrest_count_dif',
    'FS_award_count_dif',
    'FS_notes',
    'DH_date',
    'DH_charge_count',
    'DH_allegation_count',
    'case_num_x',
    'charge_y',
    'case_value_x',
    'disposition_y',
    'aa_felony_arrest_count',
    'aa_infraction_arrest_count',
    'aa_misdemeanor_arrest_count',
    'aa_other_arrest_count',
    'aa_violation_arrest_count',
    'date',
    'award',
    'case_num_y',
    'allegation',
    'case_value_y',
    'recommendation']

# Rename charges_clean variables
draft_demographic_merge = draft_demographic_merge.rename(columns={
    'case_num_x': 'CC_casenum',
    'charge_y': 'CC_charge',
    'case_value_x': 'CC_case_value',
    'disposition_y': 'CC_disposition'
})

column_names_list = draft_demographic_merge.columns.tolist()
column_names_list

```




```

charge #5: INFRACTION AMOUNT ,
'PF_command',
'PF_dept_recognitions',
'PF_rank',
'PF_shield',
'PF_appt_date',
'PF_total_arrests',
'PF_lname_fname',
'PF_race',
'DA_command',
'DA_charge',
'DA_casenum',
'DA_disposition',
'DA_penalty',
'DA_X8',
'RS_rank',
'RS_date',
'RS_shield',
'FS_lname_fname',
'FS_first_name',
'FS_middle_name',
'FS_last_name',
'FS_rank',
'FS_command',
'FS_shield',
'FS_appt_date',
'FS_total_arrests',
'FS_dept_recognitions',
'FS_ethnicity',
'FS_in_sample',
'FS_same_as_web_app',
'FS_arrest_count_dif',
'FS_award_count_dif',
'FS_notes',
'DH_date',
'DH_charge_count',
'DH_allegation_count',
'CC_casenum',
'CC_charge',
'CC_case_value',
'CC_disposition',
'aa_felony_arrest_count',
'aa_infraction_arrest_count',
'aa_misdemeanor_arrest_count',
'aa_other_arrest_count',
'aa_violation_arrest_count',
'date',
'award',
'case_num_y',
'allegation',
'case_value_y',
'recommendation']

```

```

# Checking if both variables are different
draft_demographic_merge.head()
# arrest_count
# FS_total_arrests

```



	Ticket_Number	Complainants_First_Name	Complainants_Last_Name	Tax_Registry_Number_Tax_ID	Agency	Con
0	190851430		PO	Soreca	953430	NYPD
1	196184708		.	Ennis	961743	NYPD
2	196241834		.	Wynno	961465	NYPD
3	196242292		.	Keniesky	964085	NYPD
4	196244840		.	Sinto	964778	NYPD

```
# Rename arrest_all variables
draft_demographic_merge = draft_demographic_merge.rename(columns={
    'arrest_count': 'AA_arrest_count',
    'arrest_type': 'AA_arrest_type'
})
```

```
# Rename ar_processed_clean variable
draft_demographic_merge = draft_demographic_merge.rename(columns={
    'date': 'AR_award_date',
    'award': 'AR_award_type'
})
```

```
column_names_list = draft_demographic_merge.columns.tolist()
column_names_list
```



```

    'rr_race',
    'DA_command',
    'DA_charge',
    'DA_casenum',
    'DA_disposition',
    'DA_penalty',
    'DA_X8',
    'RS_rank',
    'RS_date',
    'RS_shield',
    'FS_lname_fname',
    'FS_first_name',
    'FS_middle_name',
    'FS_last_name',
    'FS_rank',
    'FS_command',
    'FS_shield',
    'FS_appt_date',
    'FS_total_arrests',
    'FS_dept_recognitions',
    'FS_ethnicity',
    'FS_in_sample',
    'FS_same_as_web_app',
    'FS_arrest_count_dif',
    'FS_award_count_dif',
    'FS_notes',
    'DH_date',
    'DH_charge_count',
    'DH_allegation_count',
    'CC_casenum',
    'CC_charge',
    'CC_case_value',
    'CC_disposition',
    'aa_felony_arrest_count',
    'aa_infraction_arrest_count',
    'aa_misdemeanor_arrest_count',
    'aa_other_arrest_count',
    'aa_violation_arrest_count',
    'AR_award_date',
    'AR_award_type',
    'case_num_y',
    'allegation',
    'case_value_y',
    'recommendation']

# Rename allegation_detail_clean variable
draft_demographic_merge = draft_demographic_merge.rename(columns={
    'case_num_y': 'AD_allegation_casenum',
    'case_value_y': 'AD_allegation_casevalue',
    'recommendation': 'AD_allegation_recommendation',
    'CC_case_value': 'CC_casevalue'
})

# Rename / identify Oath Columns
draft_demographic_merge = draft_demographic_merge.rename(columns={
    'Violation Date': 'O_Violation_Date',
    'Violation Time': 'O_Violation_Time',
    'Issuing Agency': 'O_Issuing_Agency',
    'Respondent First Name': 'O_Respondent_first_name',
    'Respondent Last Name': 'O_Respondent_last_name',
    'Balance Due': 'O_Balance_Due',
    'Violation Location (Borough)': 'O_Violation_Location_Borough',
    'Violation Location (Block No.)': 'O_Violation_Location_Block_No.',
    'Violation Location (Lot No.)': 'O_Violation_Location_Lot_No',
    'Violation Location (House #)': 'O_Violation_Location_House_No.',
    'Violation Location (Street Name)': 'O_Violation_Location_Street_Name',
    'Violation Location (City)': 'O_Violation_Location_City',
    'Violation Location (Zip Code)': 'O_Violation_Location_Zip_Code',
    'Violation Location (State Name)': 'O_Violation_Location_State_Name',
    'Respondent Address (Borough)': 'O_Respondent_Address_Borough',

```

```

'Respondent Address (House #)': 'O_Respondent_Address_House_No.',
'Respondent Address (Street Name)': 'O_Respondent_Address_Street_Name',
'Respondent Address (City)': 'O_Respondent_Address_City',
'Respondent Address (Zip Code)': 'O_Respondent_Address_Zip_Code',
'Respondent Address (State Name)': 'O_Respondent_Address_State_Name',
'Hearing Status': 'O_Hearing_Status',
'Hearing Result': 'O_Hearing_Result',
'Scheduled Hearing Location': 'O_Scheduled_Hearing_Location',
'Hearing Date': 'O_Hearing_Date',
'Hearing Time': 'O_Hearing_Time',
'Decision Location (Borough)': 'O_Decision_Location_Borough',
'Decision Date': 'O_Decision_Date',
'Total Violation Amount': 'O_Total_Violation_Amount',
'Violation Details': 'O_Violation_Details',
'Date Judgment Docketed': 'O_Date_Judgment_Docketed',
'Respondent Address or Facility Number(For FDNY and DOB Tickets)': 'O_Respondent_Address_or_Facility_Number',
'Penalty Imposed': 'O_Penalty_Imposed',
'Paid Amount': 'O_Paid_Amount',
'Additional Penalties or Late Fees': 'O_Add_Penalties_Late_Fees',
'Compliance Status': 'O_Compliance_Status',
'Charge #1: Code': 'O_Charge1_Code',
'Charge #1: Code Section': 'O_Charge1_Code_Section',
'Charge #1: Code Description': 'O_Charge1_Code_Description',
'Charge #1: Infraction Amount': 'O_Charge1_Infraction_Amount',
'Charge #2: Code': 'O_Charge2_Code',
'Charge #2: Code Section': 'O_Charge2_Code_Section',
'Charge #2: Code Description': 'O_Charge2_Code_Description',
'Charge #2: Infraction Amount': 'O_Charge2_Infraction_Amount',
'Charge #3: Code': 'O_Charge3_Code',
'Charge #3: Code Section': 'O_Charge3_Code_Section',
'Charge #3: Code Description': 'O_Charge3_Code_Description',
'Charge #3: Infraction Amount': 'O_Charge3_Infraction_Amount'
})

```

```

column_names_list = draft_demographic_merge.columns.tolist()
column_names_list

```



```

    'FS_total_arrests',
    'FS_dept_recognitions',
    'FS_ethnicity',
    'FS_in_sample',
    'FS_same_as_web_app',
    'FS_arrest_count_dif',
    'FS_award_count_dif',
    'FS_notes',
    'DH_date',
    'DH_charge_count',
    'DH_allegation_count',
    'CC_casenum',
    'CC_charge',
    'CC_casevalue',
    'CC_disposition',
    'aa_felony_arrest_count',
    'aa_infraction_arrest_count',
    'aa_misdemeanor_arrest_count',
    'aa_other_arrest_count',
    'aa_violation_arrest_count',
    'AR_award_date',
    'AR_award_type',
    'AD_allegation_casenum',
    'allegation',
    'AD_allegation_casevalue',
    'AD_allegation_recommendation']

draft_demographic_merge = draft_demographic_merge.rename(columns={
    'allegation': 'AD_allegation_type'
})

# Filling Na with "."
draft_demographic_merge.fillna(".", inplace=True)

# Saving .csv to Google Drive
#from google.colab import drive
#drive.mount('/content/drive')

#draft_demographic_merge.to_csv('/content/drive/My Drive/draft_demographic_merge.csv', index=False)

# Assuming column_names_list is already defined
#df_columns = pd.DataFrame(column_names_list, columns=['Column Names'])
#df_columns.to_csv('column_names2.csv', index=False)

#from google.colab import files

#files.download('column_names2.csv')

#draft_demographic_merge.to_csv('draft_demographic_merge2.csv', index=False)
#from google.colab import files

#files.download('draft_demographic_merge2.csv')

# PF_command
# DA_command
# FS_command

#column_names_list = draft_demographic_merge.columns.tolist()
#column_names_list

```

```
# Just checking for any cleaning leakage before consolidation: No Missing Values

# Count of "NaN" per column
#total_nan_count = draft_demographic_merge.isna().sum()

#total_nan_count = total_nan_count.reset_index()
#total_nan_count.columns = ["Column", "NaN_Counts"]

# Optional: sort by most missing
#total_nan_count = total_nan_count.sort_values(by="NaN_Counts", ascending=False).reset_index(drop=True)
#total_nan_count

# Count of "." per column
total_dot_count = (draft_demographic_merge == ".").sum()

# Convert to a DataFrame for clean display
total_dot_count = total_dot_count.reset_index()
total_dot_count.columns = ["Column", "Dot_Counts"]

# Optional: sort by most missing
total_dot_count = total_dot_count.sort_values(by="Dot_Counts", ascending=False).reset_index(drop=True)
total_dot_count
```



	Column	Dot_Counts	
0	FS_arrest_count_dif	4663	
1	FS_award_count_dif	4663	
2	FS_notes	4663	
3	FS_same_as_web_app	4659	
4	O_Charge3_Code_Description	4647	
...	...	...	
111	aa_felony_arrest_count	0	
112	aa_misdemeanor_arrest_count	0	
113	aa_infraction_arrest_count	0	
114	aa_other_arrest_count	0	
115	aa_violation_arrest_count	0	

116 rows × 2 columns

Next steps:

[Generate code with total_dot_count](#)
[View recommended plots](#)
[New interactive sheet](#)

Start coding or [generate](#) with AI.

✓ PF, DA, FS Command Consolidation

Not using DA_command (only 19 complete values)

- For officers with both commands filled; Keep all identical PF and FS commands
- For officers with both commands filled: Rely on PF commands for dissimilar commands.
- For an officer with only one filled, rely on the filled one
- If neither filled, mark as "."

```
# Check how many non-null and non"." values exist in each command column
# PF most full
```

```
for col in ["PF_command", "DA_command", "FS_command"]:
    non_empty = draft_demographic_merge[col].ne(".").sum()
    print(f"{col}: {non_empty} complete values")
```

```
PF_command: 4513 complete values
DA_command: 19 complete values
FS_command: 4472 complete values
```

```
# Make sure taxnum is string for consistency
draft_demographic_merge["Ticket_Number"] = draft_demographic_merge["Ticket_Number"].astype(str)
```

```
# Get taxnums where each command column is non-empty
pf_ticketnums = set(draft_demographic_merge.loc[draft_demographic_merge["PF_command"] != ".", "Ticket_Number"])
da_ticketnums = set(draft_demographic_merge.loc[draft_demographic_merge["DA_command"] != ".", "Ticket_Number"])
fs_ticketnums = set(draft_demographic_merge.loc[draft_demographic_merge["FS_command"] != ".", "Ticket_Number"])
```

```
# DA_command can be dropped because all of their commands are in PF_command.
# Check subset relationships
print("Are all FS_command taxnums in PF_command?")
print(fs_ticketnums.issubset(pf_ticketnums))
```

```
print("Are all DA_command taxnums in PF_command?")
print(da_ticketnums.issubset(pf_ticketnums))
```

```
Are all FS_command taxnums in PF_command?
False
Are all DA_command taxnums in PF_command?
True
```

```
# DA_command can be dropped because all of their commands are in PF_command.
```

```
# Filter to rows where PF_command or FS_command is not "."
pf_fs_command_comparison = draft_demographic_merge[
    (draft_demographic_merge["PF_command"] != ".") |
    (draft_demographic_merge["FS_command"] != ".")
]
```

```
# Select relevant columns
pf_fs_command_comparison = pf_fs_command_comparison[["PF_command", "FS_command"]]
```

```
# Preview
pf_fs_command_comparison.head()
```

```
PF_command  FS_command
0  046 PRECINCT  046 PRECINCT
1  070 PRECINCT  070 PRECINCT
2  101 PRECINCT  101 PRECINCT
3  101 PRECINCT  101 PRECINCT
4  101 PRECINCT  101 PRECINCT
```

Next steps: [Generate code with pf_fs_command_comparison](#)

[View recommended plots](#)

[New interactive sheet](#)

```
# PF_command and FS_command Stats ~
total_rows = draft_demographic_merge.shape[0]
print(f"Total Ticket_Numbers: {total_rows}")
```

```

print("Total Ticket_Numbers with PF_command:", (draft_demographic_merge["PF_command"] != ".").sum())
print("Total Ticket_Numbers with FS_command:", (draft_demographic_merge["FS_command"] != ".").sum())
# Officers with no associated commands (both PF_command and FS_command are ".")
no_command = draft_demographic_merge[
    (draft_demographic_merge["PF_command"] == ".") &
    (draft_demographic_merge["FS_command"] == ".")
]

# Show how many
print(f"Ticket_Numbers missing both PF and FS: {no_command.shape[0]}")

# View their Ticket Numbers
no_command[["Ticket_Number", "Final_Officer_Tax_ID", "PF_command", "FS_command"]].head()

# 1. Total Officers with only 1 command filled:
only_one_filled = draft_demographic_merge[
    ((draft_demographic_merge["PF_command"] != ".") & (draft_demographic_merge["FS_command"] == ".")) |
    ((draft_demographic_merge["PF_command"] == ".") & (draft_demographic_merge["FS_command"] != "."))
]

print(f"Total Ticket_Numbers with only one command filled: {only_one_filled.shape[0]}")

# 2. How many officers have both PF_command and FS_command filled?
both_filled = draft_demographic_merge[
    (draft_demographic_merge["PF_command"] != ".") &
    (draft_demographic_merge["FS_command"] != ".")
]

print(f"Total Ticket_Numbers with both PF_command and FS_command filled: {both_filled.shape[0]}")

# 3. Among those officers with both filled, how many have identical vs dissimilar commands?

# Create a boolean column temporarily
both_filled["commands_match"] = both_filled["PF_command"] == both_filled["FS_command"]

# Count matches and mismatches
identical_commands = both_filled["commands_match"].sum() # True = 1
dissimilar_commands = (~both_filled["commands_match"]).sum() # False = 1

print(f"Ticket_Numbers with identical PF and FS commands: {identical_commands} / {both_filled.shape[0]}")
print(f"Ticket_Numbers with dissimilar PF and FS commands: {dissimilar_commands} / {both_filled.shape[0]}")

```

```

🔗 Total Ticket_Numbers: 4663
Total Ticket_Numbers with PF_command: 4513
Total Ticket_Numbers with FS_command: 4472
Ticket_Numbers missing both PF and FS: 35
Total Ticket_Numbers with only one command filled: 271
Total Ticket_Numbers with both PF_command and FS_command filled: 4357
Ticket_Numbers with identical PF and FS commands: 3034 / 4357
Ticket_Numbers with dissimilar PF and FS commands: 1323 / 4357

```

```

# Solution:
# - For officers with both commands filled; Keep all identical PF and FS commands
# - For officers with both commands filled: Rely on FS commands for dissimilar commands
# - For officer with only one filled: rely on filled one
# - If neither filled, mark as "."

```

```

# Solution function:

```

```

def get_master_command(row):
    pf = row["PF_command"]
    fs = row["FS_command"]

    # Both commands filled
    if pf != "." and fs != ".":
        if pf == fs:
            return pf # Identical - keep PF (or FS)
        else:
            return fs # Dissimilar - keep FS (or PF)
    # Only one command filled
    if pf != ".":
        return pf
    if fs != ".":
        return fs
    # Neither filled
    return "."

```



```

        return pf # identical = keep PF (or FS)
    else:
        return fs # Dissimilar = use prioritize PF

# Only PF filled
elif pf != ".":
    return pf # Only PF exists = keep PF

# Only FS filled
elif fs != ".":
    return fs # Only FS exists = keep FS

# Neither filled
else:
    return "." # Neither filled = mark as "."

# Apply function
draft_demographic_merge["Master_Command"] = draft_demographic_merge.apply(get_master_command, axis=1)

draft_demographic_merge[["Ticket_Number", "Final_Officer_Tax_ID", "PF_command", "FS_command", "Master_Command"]]

```

	Ticket_Number	Final_Officer_Tax_ID	PF_command	FS_command	Master_Command
0	190851430	953430	046 PRECINCT	046 PRECINCT	046 PRECINCT
1	196184708	961743	070 PRECINCT	070 PRECINCT	070 PRECINCT
2	196241834	961465	101 PRECINCT	101 PRECINCT	101 PRECINCT
3	196242292	964085	101 PRECINCT	101 PRECINCT	101 PRECINCT
4	196244840	964778	101 PRECINCT	101 PRECINCT	101 PRECINCT
5	196264925	963591	105 PRECINCT	105 PRECINCT	105 PRECINCT
6	196269581	963656	105 PRECINCT	105 PRECINCT	105 PRECINCT
7	196320722	952904	125 STREET TASK FORCE	MILITARY & EXTENDED LEAVE DESK	MILITARY & EXTENDED LEAVE DESK
8	196374613	966902	032 PRECINCT	YOUTH STRATEGIES DIVISION	YOUTH STRATEGIES DIVISION
9	196380892	966375	108 PRECINCT	.	108 PRECINCT
10	196384421	966645	079 PRECINCT	079 PRECINCT	079 PRECINCT
11	196384440	964591	079 PRECINCT	079 PRECINCT	079 PRECINCT
12	196384843	966613	079 PRECINCT	.	079 PRECINCT
13	196384861	966613	079 PRECINCT	.	079 PRECINCT
14	196385907	966888	.	.	.
15	196385925	966858	079 PRECINCT	079 PRECINCT	079 PRECINCT
16	196386530	966441	079 PRECINCT	079 PRECINCT	079 PRECINCT
17	196386960	966548	079 PRECINCT	079 PRECINCT	079 PRECINCT
18	196387080	966728	079 PRECINCT	079 PRECINCT	079 PRECINCT
19	196396018	959907	088 PRECINCT	088 PRECINCT	088 PRECINCT

Checking

```

# 1. Check how many rows have a valid Master_Command (not ".")
total_rows = draft_demographic_merge.shape[0]
valid_master_commands = (draft_demographic_merge["Master_Command"] != ".").sum()

print(f"Total Ticket_Numbers: {total_rows}")
print(f"Ticket_Numbers with a valid Master_Command: {valid_master_commands}")

```

```
# 2. Check that when both PF and FS are filled, the Master_Command matches PF_command
both_filled = draft_demographic_merge[
    (draft_demographic_merge["PF_command"] != ".") &
    (draft_demographic_merge["FS_command"] != ".")
]

# How many rows where Master_Command == PF_command?
pf_priority_correct = (both_filled["Master_Command"] == both_filled["PF_command"]).sum()

print(f"Among Ticket_Numbers with both commands filled, Master_Command matches PF_command in {pf_priority_correc

# 3. Check how many officers are missing both PF and FS (sanity that Master_Command is ".")
both_missing = draft_demographic_merge[
    (draft_demographic_merge["PF_command"] == ".") &
    (draft_demographic_merge["FS_command"] == ".")
]

missing_master_command = (both_missing["Master_Command"] == ".").sum()

print(f"Ticket_Numbers missing both PF and FS: {both_missing.shape[0]}")
print(f"Ticket_Numbers correctly marked as missing in Master_Command: {missing_master_command}")
```

```
↗ Total Ticket_Numbers: 4663
Ticket_Numbers with a valid Master_Command: 4628
Among Ticket_Numbers with both commands filled, Master_Command matches PF_command in 3034 out of 4357 cases
Ticket_Numbers missing both PF and FS: 35
Ticket_Numbers correctly marked as missing in Master_Command: 35
```

✓ PF, FS, RS Rank Consolidation

- Master_Rank & Master_Rank_Date: most recent rank and date

```
draft_demographic_merge[["PF_rank", "FS_rank", "RS_rank", "PF_appt_date", "FS_appt_date", "RS_date"]].head(5)
```

	PF_rank	FS_rank	RS_rank	PF_appt_date	FS_appt_date	RS_date	
0	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	07/11/12 12:00 AM	07/11/2012 00:00:00	[2012-07-11 00:00:00]	
1	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	07/13/16 12:00 AM	07/13/2016 00:00:00	[2016-07-13 00:00:00]	
2	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	01/06/16 12:00 AM	01/06/2016 00:00:00	[2016-01-06 00:00:00]	
3	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	07/05/17 12:00 AM	07/05/2017 00:00:00	[2017-07-05 00:00:00]	
4	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	10/18/17 12:00 AM	10/18/2017 00:00:00	[2017-10-18 00:00:00]	

```
# All times 12:00AM and 00:00:00 are consistent for all entries; focus on dates
draft_demographic_merge["PF_appt_date"] = pd.to_datetime(draft_demographic_merge["PF_appt_date"], errors='coerce')
draft_demographic_merge["FS_appt_date"] = pd.to_datetime(draft_demographic_merge["FS_appt_date"], errors='coerce')
draft_demographic_merge["RS_date"] = draft_demographic_merge["RS_date"].apply(lambda x: x[0] if isinstance(x, list) else x)
draft_demographic_merge["RS_date"] = pd.to_datetime(draft_demographic_merge["RS_date"], errors='coerce').dt.date
```

```
draft_demographic_merge[["PF_rank"]].info()
```

```
↗ <class 'pandas.core.frame.DataFrame'>
RangeIndex: 4663 entries, 0 to 4662
Data columns (total 1 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    PF_rank     4663 non-null   object
dtypes: object(1)
memory usage: 36.6+ KB
```

```

# Officers where at least two rank sources are filled
rank_filled = draft_demographic_merge[
    (draft_demographic_merge["PF_rank"] != ".") |
    (draft_demographic_merge["FS_rank"] != ".") |
    (draft_demographic_merge["RS_rank"] != ".")
]

# Function to check if all non-missing ranks match
def ranks_match(row):
    ranks = []
    for col in ["PF_rank", "FS_rank", "RS_rank"]:
        if row[col] != ".":
            ranks.append(row[col].strip())
    unique_ranks = set(ranks)

    if len(unique_ranks) <= 1:
        return True # All ranks are the same (or only one rank exists)
    else:
        return False # Multiple different ranks

# Apply it
rank_filled["Ranks_Match"] = rank_filled.apply(ranks_match, axis=1)

# Count
matching_ranks = rank_filled["Ranks_Match"].sum()
non_matching_ranks = (~rank_filled["Ranks_Match"]).sum()

print(f"Total rows with at least one rank filled: {rank_filled.shape[0]}")
print(f"Officers with matching ranks: {matching_ranks}")
print(f"Officers with dissimilar ranks: {non_matching_ranks}")

```

 Total rows with at least one rank filled: 4628
 Officers with matching ranks: 4280
 Officers with dissimilar ranks: 348

```

def get_master_rank_and_date(row):
    # Create a list to hold (rank, date) pairs
    rank_dates = []

    # Add PF_rank if available
    if row["PF_rank"] != "." and row["PF_appt_date"] != ".":
        rank_dates.append((row["PF_rank"], row["PF_appt_date"]))

    # Add FS_rank if available
    if row["FS_rank"] != "." and row["FS_appt_date"] != ".":
        rank_dates.append((row["FS_rank"], row["FS_appt_date"]))

    # Add RS_rank if available
    if row["RS_rank"] != "." and row["RS_date"] != ".":
        rank_dates.append((row["RS_rank"], row["RS_date"]))

    # No valid rank-date pairs found
    if not rank_dates:
        return pd.Series([".", "."])

    # Find the most recent (by date)
    most_recent = max(rank_dates, key=lambda x: x[1])

    # Return the corresponding rank and date
    return pd.Series([most_recent[0], most_recent[1]])

# Apply it
draft_demographic_merge[["master_rank", "master_rank_date"]] = draft_demographic_merge.apply(get_master_rank_ar

draft_demographic_merge[["PF_rank", "FS_rank", "RS_rank", "PF_appt_date", "FS_appt_date", "RS_date", "master_ra

```

	PF_rank	FS_rank	RS_rank	PF_appt_date	FS_appt_date	RS_date	master_rank	master_rank_date	
0	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	2012-07-11	2012-07-11	2012-07-11	POLICE OFFICER	2012-07-11	
1	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	2016-07-13	2016-07-13	2016-07-13	POLICE OFFICER	2016-07-13	
2	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	2016-01-06	2016-01-06	2016-01-06	POLICE OFFICER	2016-01-06	
3	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	2017-07-05	2017-07-05	2017-07-05	POLICE OFFICER	2017-07-05	
4	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	2017-10-18	2017-10-18	2017-10-18	POLICE OFFICER	2017-10-18	
...	...	...	...	...	...	...	...	...	
95	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	2018-01-10	2018-01-10	2018-01-10	POLICE OFFICER	2018-01-10	
96	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	2007-07-09	2007-07-09	2007-07-09	POLICE OFFICER	2007-07-09	
97	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	2019-04-15	2019-04-15	2019-04-15	POLICE OFFICER	2019-04-15	
98	POLICE OFFICER	POLICE OFFICER	POLICE OFFICER	2019-04-15	2019-04-15	2019-04-15	POLICE OFFICER	2019-04-15	
99	POLICE OFFICER	SERGEANT	SERGEANT	2011-07-06	2011-07-06	2022-02-09	SERGEANT	2022-02-09	

100 rows x 8 columns

```

# Define the ranks_match function
def ranks_match(row):
    ranks = []
    for col in ["PF_rank", "FS_rank", "RS_rank"]:
        if row[col] != ".":
            ranks.append(row[col].strip())
    unique_ranks = set(ranks)

    return len(unique_ranks) <= 1 # True if all ranks match (or only one filled), False if multiple differ

# Apply it
draft_demographic_merge["Ranks_Match"] = draft_demographic_merge.apply(ranks_match, axis=1)

# Filter officers with dissimilar ranks
dissimilar_ranks = draft_demographic_merge[draft_demographic_merge["Ranks_Match"] == False]

# Select relevant columns
dissimilar_ranks_dates = dissimilar_ranks[
    ["Final_Officer_Tax_ID", "PF_rank", "PF_appt_date", "FS_rank", "FS_appt_date", "RS_rank", "RS_date"]
]

# View the first few
dissimilar_ranks_dates.head(10)
#dissimilar_ranks_dates.describe()
# Total dissimilar ranks = 454

```

	Final_Officer_Tax_ID	PF_rank	PF_appt_date	FS_rank	FS_appt_date	RS_rank	RS_date	
25	951992	POLICE OFFICER	2012-01-09	POLICE OFFICER	2012-01-09	SERGEANT	2022-06-24	
29	948461	POLICE OFFICER	2010-01-11	POLICE OFFICER	2010-01-11	SERGEANT	2022-05-27	
99	949956	POLICE OFFICER	2011-07-06	SERGEANT	2011-07-06	SERGEANT	2022-02-09	
136	952889	POLICE OFFICER	2012-07-11	SERGEANT	2012-07-11	SERGEANT	2022-02-09	
137	952889	POLICE OFFICER	2012-07-11	SERGEANT	2012-07-11	SERGEANT	2022-02-09	
153	927362	DETECTIVE SPECIALIST	2000-09-29	DETECTIVE SPECIALIST	2000-09-29	DETECTIVE	2009-08-05	
154	927362	DETECTIVE SPECIALIST	2000-09-29	DETECTIVE SPECIALIST	2000-09-29	DETECTIVE	2009-08-05	
201	935192	POLICE OFFICER	2004-07-01	DETECTIVE SPECIALIST	2004-07-01	DETECTIVE	2022-05-27	
223	951466	POLICE OFFICER	2011-07-06	POLICE OFFICER	2011-07-06	SERGEANT	2022-05-27	
339	949763	POLICE OFFICER	2010-07-06	SERGEANT	2010-07-06	SERGEANT	2022-02-09	

```

# Checking to see
draft_demographic_merge.loc[
    draft_demographic_merge["Final_Officer_Tax_ID"] == "945614",
    ["master_rank", "master_rank_date", "RS_rank", "RS_date"]
]

```

	master_rank	master_rank_date	RS_rank	RS_date	
133	CAPTAIN	2018-09-28	CAPTAIN	2018-09-28	

```
# Checking to see
draft_demographic_merge.loc[
    draft_demographic_merge["Final_Officer_Tax_ID"] == "927362",
    ["master_rank", "master_rank_date", "RS_rank", "RS_date"]
]
```



	master_rank	master_rank_date	RS_rank	RS_date
153	DETECTIVE	2009-08-05	DETECTIVE	2009-08-05
154	DETECTIVE	2009-08-05	DETECTIVE	2009-08-05



Double-click (or enter) to edit

```
draft_demographic_merge.master_rank.unique()
```



```
array(['POLICE OFFICER', '.', 'SERGEANT', 'CAPTAIN', 'DETECTIVE',
      'INSPECTOR', 'LIEUTENANT'], dtype=object)
```

```
# Check how many entries have multiple RS dates
```

```
multiple_rs_dates = draft_demographic_merge["RS_date"].apply(lambda x: isinstance(x, list) and len(x) > 1)
```

```
# Count how many rows this applies to
```

```
print("Officers with multiple RS_dates:", multiple_rs_dates.sum())
```

```
# Optional: view them
```

```
draft_demographic_merge[multiple_rs_dates][["Final_Officer_Tax_ID", "RS_rank", "RS_date"]].head()
```



```
Officers with multiple RS_dates: 0
```

Final_Officer_Tax_ID	RS_rank	RS_date
----------------------	---------	---------



```
# First, how many total rows
```

```
total_rows = draft_demographic_merge.shape[0]
```

```
# How many non-empty values in Master_Rank (not "." and not NaN)
```

```
master_rank_complete = draft_demographic_merge["master_rank"].ne(".").sum()
```

```
# Print the results
```

```
print(f"Total rows: {total_rows}")
```

```
print(f"master_rank complete entries: {master_rank_complete}")
```



```
Total rows: 4663
master_rank complete entries: 4628
```

```
# These are Ticket's and Tax ID's that have a missing rank in all PF, FS, RS: What should I do with these?
```

```
# Subset rows where Master_Rank is missing (".")
```

```
missing_master_rank = draft_demographic_merge[draft_demographic_merge["master_rank"] == "."]
```

```
# How many missing?
```

```
print(f"Number of missing Master_Rank entries: {missing_master_rank.shape[0]}")
```

```
# Preview the missing ones
```

```
missing_master_rank[["Ticket_Number", "Final_Officer_Tax_ID", "PF_rank", "FS_rank", "RS_rank", "master_rank"]].
```

➦ Number of missing Master_Rank entries: 35

	Ticket_Number	Final_Officer_Tax_ID	PF_rank	FS_rank	RS_rank	master_rank	
14	196385907	966888	.	.	.	.	
20	196403498	960914	.	.	.	.	
80	197998671	957044	.	.	.	.	
108	198345988	957091	.	.	.	.	
116	198423592	923153	.	.	.	.	
183	198973390	964321	.	.	.	.	
413	200264278	967441	.	.	.	.	
432	200343056	950415	.	.	.	.	
594	202110352	958908	.	.	.	.	
765	208110550	917525	.	.	.	.	

✓ * PF, RS, FS Shield Consolidation *

- PF_shield: represent "other shields" on <https://legalaidnyc.org/law-enforcement-look-up/>
- FS_shield: represents "current shield" on <https://legalaidnyc.org/law-enforcement-look-up/>
-

```
draft_demographic_merge[["Ticket_Number", "Final_Officer_Tax_ID", "PF_shield", "FS_shield", "RS_shield"]].head()
```

➦

	Ticket_Number	Final_Officer_Tax_ID	PF_shield	FS_shield	RS_shield	
0	190851430	953430	5884.0	5884.0	5884	
1	196184708	961743	24053.0	24053.0	24053	
2	196241834	961465	18344.0	18344.0	18344	
3	196242292	964085	26036.0	26036.0	26036	
4	196244840	964778	4519.0	4519.0	4519	
5	196264925	963591	8697.0	8697.0	8697	
6	196269581	963656	9789.0	9789.0	9789	
7	196320722	952904	13650.0	13650.0	13650	
8	196374613	966902	3355.0	3355.0	3355	
9	196380892	966375	6042.0	.	.	

```
# Which file has the most shield values?
```

```
shield_cols = ["PF_shield", "FS_shield", "RS_shield"]
```

```
# Count how many entries are not "." in each column
```

```
for col in shield_cols:
```

```
    count = (draft_demographic_merge[col] != ".").sum()
```

```
    print(f"{col}: {count} complete shield numbers")
```

```
# PF_shield has the most shield number.
```

➦ PF_shield: 4510 complete shield numbers
 FS_shield: 4469 complete shield numbers
 RS_shield: 4469 complete shield numbers

```
# Making all shield values into int to be comparable
def standardize_shield(val):
    if val == "." or pd.isna(val):
        return "." # Preserve explicit missing values
    try:
        return int(float(val)) # Handles strings like '5884.0' or floats like 5884.0
    except:
        return "."

for col in ["PF_shield", "FS_shield", "RS_shield"]:
    draft_demographic_merge[col] = draft_demographic_merge[col].apply(standardize_shield)
```

```
draft_demographic_merge[["Ticket_Number", "Final_Officer_Tax_ID", "PF_shield", "FS_shield", "RS_shield"]].head(10)
```

	Ticket_Number	Final_Officer_Tax_ID	PF_shield	FS_shield	RS_shield
0	190851430	953430	5884	5884	5884
1	196184708	961743	24053	24053	24053
2	196241834	961465	18344	18344	18344
3	196242292	964085	26036	26036	26036
4	196244840	964778	4519	4519	4519
5	196264925	963591	8697	8697	8697
6	196269581	963656	9789	9789	9789
7	196320722	952904	13650	13650	13650
8	196374613	966902	3355	3355	3355
9	196380892	966375	6042	.	.

```
# Compare shield values across all three files. Are there any discrepancies?
draft_demographic_merge["valid_shield_count"] = draft_demographic_merge[["PF_shield", "FS_shield", "RS_shield"]
    lambda row: sum(cell != "." for cell in row), axis=1
)
```

```
shield_multi_filled = draft_demographic_merge[draft_demographic_merge["valid_shield_count"] >= 2]
```

```
def shields_match(row):
    shields = [row[col] for col in ["PF_shield", "FS_shield", "RS_shield"] if row[col] != "."]
    return len(set(shields)) == 1
shield_multi_filled["shields_match"] = shield_multi_filled.apply(shields_match, axis=1)
```

```
dissimilar_shields = shield_multi_filled[shield_multi_filled["shields_match"] == False]
```

```
# Summary
print(f"{len(dissimilar_shields)} rows have shield values that do not match.")
```

```
# 4321/4663 are similar
```

```
342 rows have shield values that do not match.
```

```
# Filter to mismatched rows
# Compared these shield numbers to https://legalaidnyc.org/law-enforcement-look-up/; most look like RS_shield is
dissimilar_shields = shield_multi_filled[shield_multi_filled["shields_match"] == False]
dissimilar_shields[["Ticket_Number", "Final_Officer_Tax_ID", "Final_Officer_Last_Name", "Final_Officer_First_Nam
```




	Ticket_Number	Final_Officer_Tax_ID	Final_Officer_Last_Name	Final_Officer_First_Name	PF_shield	FS_
	25	196440292	951992	Mcleod	Russel	10181
	29	196464759	948461	Blackman	Justin	7785
	41	196616081	939119	Mattocks	William	29334
	99	198171775	949956	Michel	Andre	14401
	136	198585916	952889	Javid	Mohsin	13649
	...	...	...	...	...	...
	4563	211915166	970041	Ohara	Dennis	18750
	4636	214540810	941927	Ignatz	Michael	19499
	4638	196430008	949047	Gonzalezalicea	Priscilla	1447
	4660	211744895	948461	Blackman	Justin	7785
	4661	211744904	948461	Blackman	Justin	7785

342 rows x 7 columns

```
rs_no_fs = dissimilar_shields[dissimilar_shields["RS_shield"] != dissimilar_shields["FS_shield"]][["Final_Office
rs_no_fs
```



	Final_Officer_Tax_ID	PF_shield	FS_shield	RS_shield	
	25	951992	10181	10181	4457
	29	948461	7785	7785	3398
	41	939119	29334	29334	29325
	223	951466	30669	30669	3731
	250	942524	2728	2728	26267
	...	...	...	...	...
	4563	970041	18750	18750	24426
	4636	941927	19499	19499	6067
	4638	949047	1447	1447	3519
	4660	948461	7785	7785	3398
	4661	948461	7785	7785	3398

212 rows x 4 columns

Start coding or [generate](#) with AI.

```
# Can officer shield numbers change? This is something to investigate.
# As for right now, we are going to rely on the FS_shield to incorporate into the Master_Shield column.

# Create Current_Shield column
# 1) Retain all identical shields;
# 2) All RS_shield that match FS_shield, go into Current_Shield
# 3) All remaining unmatched shields from PF_shield , go into Other_Shield column

def assign_current_shield(row):
    pf, fs, rs = row["PF_shield"], row["FS_shield"], row["RS_shield"]

    # If all valid and the same: assign
    shields = [s for s in [pf, fs, rs] if s != ""]
```

```
if len(set(shields)) == 1:
    return pd.Series([shields[0], "."])

# If FS and RS match: trust FS/RS, put PF in Other
if fs != "." and rs != "." and str(fs) == str(rs):
    return pd.Series([fs, pf if pf != fs else "."])

# Else → fallback: use PF as Other
return pd.Series([".", pf])

draft_demographic_merge[["Current_Shield", "Other_Shield"]] = draft_demographic_merge.apply(assign_current_shie

draft_demographic_merge[["Final_Officer_Tax_ID", "PF_shield", "FS_shield", "RS_shield", "Current_Shield", "Othe
```



	Final_Officer_Tax_ID	PF_shield	FS_shield	RS_shield	Current_Shield	Other_Shield	
0	953430	5884	5884	5884	5884	.	
1	961743	24053	24053	24053	24053	.	
2	961465	18344	18344	18344	18344	.	
3	964085	26036	26036	26036	26036	.	
4	964778	4519	4519	4519	4519	.	
5	963591	8697	8697	8697	8697	.	
6	963656	9789	9789	9789	9789	.	
7	952904	13650	13650	13650	13650	.	
8	966902	3355	3355	3355	3355	.	
9	966375	6042	.	.	6042	.	

```
draft_demographic_merge[
    draft_demographic_merge["Final_Officer_Tax_ID"].isin(["951992", "948461", "939119"])
][
    ["Final_Officer_Tax_ID", "PF_shield", "FS_shield", "RS_shield", "Current_Shield", "Other_Shield"]
]
```



	Final_Officer_Tax_ID	PF_shield	FS_shield	RS_shield	Current_Shield	Other_Shield
25	951992	10181	10181	4457	.	10181
29	948461	7785	7785	3398	.	7785
41	939119	29334	29334	29325	.	29334
1655	948461	7785	7785	3398	.	7785
1656	948461	7785	7785	3398	.	7785
1657	948461	7785	7785	3398	.	7785
1658	948461	7785	7785	3398	.	7785
1659	948461	7785	7785	3398	.	7785
1660	948461	7785	7785	3398	.	7785
1661	948461	7785	7785	3398	.	7785
2342	948461	7785	7785	3398	.	7785
2510	948461	7785	7785	3398	.	7785
2783	948461	7785	7785	3398	.	7785
2939	948461	7785	7785	3398	.	7785
3191	948461	7785	7785	3398	.	7785
4660	948461	7785	7785	3398	.	7785
4661	948461	7785	7785	3398	.	7785



✓ PF and FS Total Arrests Consolidation

- FS_total_arrests will be relied on (pulled in 2022)
- If no values in FS_total_arrests rely on PF_total_arrests

```
draft_demographic_merge[["Ticket_Number", "Final_Officer_Tax_ID", "PF_total_arrests", "FS_total_arrests"]].head
```



	Ticket_Number	Final_Officer_Tax_ID	PF_total_arrests	FS_total_arrests
0	190851430	953430	102.0	115.0
1	196184708	961743	49.0	63.0
2	196241834	961465	50.0	63.0
3	196242292	964085	26.0	35.0
4	196244840	964778	74.0	80.0
5	196264925	963591	31.0	53.0
6	196269581	963656	33.0	45.0
7	196320722	952904	258.0	260.0
8	196374613	966902	50.0	56.0
9	196380892	966375	28.0	.



```
# Which file has the most shield values?
total_arrests_cols = ["PF_total_arrests", "FS_total_arrests"]
```

```
# Count how many entries are not "." in each column
for col in total_arrests_cols:
    count = (draft_demographic_merge[col] != ".").sum()
```

```
print(f"{col}: {count} complete total_arrests rows")
```

```
PF_total_arrests: 4513 complete total_arrests rows
FS_total_arrests: 4472 complete total_arrests rows
```

```
# Making total_arrests column
def choose_total_arrests(row):
    if row["FS_total_arrests"] != ".":
        return row["FS_total_arrests"]
    else:
        return row["PF_total_arrests"]
```

```
draft_demographic_merge["total_arrests"] = draft_demographic_merge.apply(choose_total_arrests, axis=1)
```

```
draft_demographic_merge[["Ticket_Number", "Final_Officer_Tax_ID", "PF_total_arrests", "FS_total_arrests", "total
```

	Ticket_Number	Final_Officer_Tax_ID	PF_total_arrests	FS_total_arrests	total_arrests	
0	190851430	953430	102.0	115.0	115.0	
1	196184708	961743	49.0	63.0	63.0	
2	196241834	961465	50.0	63.0	63.0	
3	196242292	964085	26.0	35.0	35.0	
4	196244840	964778	74.0	80.0	80.0	
5	196264925	963591	31.0	53.0	53.0	
6	196269581	963656	33.0	45.0	45.0	
7	196320722	952904	258.0	260.0	260.0	
8	196374613	966902	50.0	56.0	56.0	
9	196380892	966375	28.0	.	28.0	

✓ PF and FS Race Consolidation

```
draft_demographic_merge[["Ticket_Number", "Final_Officer_Tax_ID", "FS_ethnicity", "PF_race"]].head(10)
```

	Ticket_Number	Final_Officer_Tax_ID	FS_ethnicity	PF_race	
0	190851430	953430	WHITE	White	
1	196184708	961743	WHITE	White	
2	196241834	961465	WHITE	White	
3	196242292	964085	WHITE	White	
4	196244840	964778	WHITE	White	
5	196264925	963591	WHITE	White	
6	196269581	963656	WHITE	White	
7	196320722	952904	BLACK	Black	
8	196374613	966902	HISPANIC	Hispanic	
9	196380892	966375	.	White	

```
# Which file has the most shield values?
total_race_cols = ["FS_ethnicity", "PF_race"]
```

```
# Count how many entries are not "." in each column
for col in total_race_cols:
    count = (draft_demographic_merge[col] != ".").sum()
    print(f"{col}: {count} complete race/ethnicity rows")
```

```
FS_ethnicity: 4448 complete race/ethnicity rows
PF_race: 4513 complete race/ethnicity rows
```

```
# Making final_race column
def final_race(row):
    if row["FS_ethnicity"] != ".":
        return row["FS_ethnicity"]
    else:
        return row["PF_race"]
```

```
draft_demographic_merge["final_race"] = draft_demographic_merge.apply(final_race, axis=1)
```

```
draft_demographic_merge[["Ticket_Number", "Final_Officer_Tax_ID", "PF_race", "FS_ethnicity", "final_race"]].head
```

	Ticket_Number	Final_Officer_Tax_ID	PF_race	FS_ethnicity	final_race
0	190851430	953430	White	WHITE	WHITE
1	196184708	961743	White	WHITE	WHITE
2	196241834	961465	White	WHITE	WHITE
3	196242292	964085	White	WHITE	WHITE
4	196244840	964778	White	WHITE	WHITE
5	196264925	963591	White	WHITE	WHITE
6	196269581	963656	White	WHITE	WHITE
7	196320722	952904	Black	BLACK	BLACK
8	196374613	966902	Hispanic	HISPANIC	HISPANIC
9	196380892	966375	White	.	White

```
# Standardize race as all caps
draft_demographic_merge.final_race.unique()
```

```
# Standardize race as all caps
draft_demographic_merge["final_race"] = draft_demographic_merge["final_race"].astype(str).str.upper()
```

```
draft_demographic_merge.final_race.unique()
```

```
array(['WHITE', 'BLACK', 'HISPANIC', 'ASIAN', '.', 'NATIVE AMERICAN'],
      dtype=object)
```

✓ PF and FS Dept. Recognition Consolidation

- Rely on FS_dept_recognitions if there are both values present; If no FS_dept_recognition, rely on PF_dept_recognition.

Double-click (or enter) to edit

```
draft_demographic_merge[["Ticket_Number", "Final_Officer_Tax_ID", "PF_dept_recognitions", "FS_dept_recognitions"]]
```



	Ticket_Number	Final_Officer_Tax_ID	PF_dept_recognitions	FS_dept_recognitions
0	190851430	953430	1.0	1.0
1	196184708	961743	0.0	0.0
2	196241834	961465	0.0	0.0
3	196242292	964085	0.0	0.0
4	196244840	964778	0.0	0.0
5	196264925	963591	0.0	1.0
6	196269581	963656	0.0	0.0
7	196320722	952904	0.0	0.0
8	196374613	966902	0.0	0.0
9	196380892	966375	4.0	.



Which file has the most shield values?

```
total_dept_recognitions_cols = ["PF_dept_recognitions", "FS_dept_recognitions"]
```

Count how many entries are not "." in each column

```
for col in total_dept_recognitions_cols:
    count = (draft_demographic_merge[col] != ".").sum()
    print(f"{col}: {count} complete total_arrests rows")
```



```
PF_dept_recognitions: 4513 complete total_arrests rows
FS_dept_recognitions: 4472 complete total_arrests rows
```

Making dept_recognitions column

```
def total_dept_recognitions(row):
    if row["FS_dept_recognitions"] != ".":
        return row["FS_dept_recognitions"]
    else:
        return row["PF_dept_recognitions"]
```

```
draft_demographic_merge["master_dept_recognitions"] = draft_demographic_merge.apply(total_dept_recognitions, a
```

```
draft_demographic_merge[["Ticket_Number", "Final_Officer_Tax_ID", "PF_dept_recognitions", "FS_dept_recognitions"
```



	Ticket_Number	Final_Officer_Tax_ID	PF_dept_recognitions	FS_dept_recognitions	master_dept_recognitions
0	190851430	953430	1.0	1.0	1.0
1	196184708	961743	0.0	0.0	0.0
2	196241834	961465	0.0	0.0	0.0
3	196242292	964085	0.0	0.0	0.0
4	196244840	964778	0.0	0.0	0.0
5	196264925	963591	0.0	1.0	1.0
6	196269581	963656	0.0	0.0	0.0
7	196320722	952904	0.0	0.0	0.0
8	196374613	966902	0.0	0.0	0.0
9	196380892	966375	4.0	.	4.0

✓ PF and FS LName, FName

- PF_lname_fname
- Rely on FS_lname_fname

```
draft_demographic_merge[["Ticket_Number", "Final_Officer_Tax_ID", "PF_lname_fname", "FS_lname_fname", "Last_Nam
```

	Ticket_Number	Final_Officer_Tax_ID	PF_lname_fname	FS_lname_fname	Last_Name_First_Name
0	190851430	953430	SORECA, NICHOLAS V	SORECA, NICHOLAS V	Soreca, Nicholas
1	196184708	961743	ENNIS, PATRICIA	ENNIS, PATRICIA	Ennis, Patricia
2	196241834	961465	WYNNE, JAMES W	WYNNE, JAMES W	Wynne, James
3	196242292	964085	KENJESKY, MICHAEL V	KENJESKY, MICHAEL V	Kenjesky, Michael
4	196244840	964778	SINTO, MICHAEL G	SINTO, MICHAEL G	Sinto, Michael
5	196264925	963591	KENNEDY, ERIN L	KENNEDY, ERIN L	Kennedy, Erin
6	196269581	963656	MONACO, MARISSA L	MONACO, MARISSA L	Dinis, Marissa
7	196320722	952904	JONES, FREDDY	JONES, FREDDY	Jones, Freddy
8	196374613	966902	VELAZQUEZ, ANTHONY F	VELAZQUEZ, ANTHONY F	Velazquez, Anthony
9	196380892	966375	VADALA, JOSEPH G	.	Vadala, Joseph

```
# Which file has the most shield values?
```

```
lname_fname = ["PF_lname_fname", "FS_lname_fname", "Last_Name_First_Name"]
```

```
# Count how many entries are not "." in each column
```

```
for col in lname_fname:
```

```
    count = (draft_demographic_merge[col] != ".").sum()
```

```
    print(f"{col}: {count} rows with full names")
```

```
PF_lname_fname: 4513 rows with full names
FS_lname_fname: 4472 rows with full names
Last_Name_First_Name: 4662 rows with full names
```

```
# Standardize names to uppercase for consistent comparison
```

```
for col in ["PF_lname_fname", "FS_lname_fname", "Last_Name_First_Name"]:
```

```
    draft_demographic_merge[col] = draft_demographic_merge[col].astype(str).str.upper()
```

```
# Identify matching rows (all three columns equal)
```

```
draft_demographic_merge["name_match_all"] = (
```

```
    (draft_demographic_merge["PF_lname_fname"] == draft_demographic_merge["FS_lname_fname"]) &
```

```
    (draft_demographic_merge["PF_lname_fname"] == draft_demographic_merge["Last_Name_First_Name"]))
```

```
)
```

```
# Summary
```

```
total_rows = draft_demographic_merge.shape[0]
```

```
matching_rows = draft_demographic_merge["name_match_all"].sum()
```

```
non_matching_rows = total_rows - matching_rows
```

```
print(f"{matching_rows} of {total_rows} rows have all three name columns matching.")
```

```
print(f"{non_matching_rows} rows have discrepancies across name columns.")
```

```
1042 of 4663 rows have all three name columns matching.
3621 rows have discrepancies across name columns.
```

```
# See discrepancies for review
name_discrepancies = draft_demographic_merge[~draft_demographic_merge["name_match_all"]][
    ["Final_Officer_Tax_ID", "PF_lname_fname", "FS_lname_fname", "Last_Name_First_Name"]
]
name_discrepancies
```



	Final_Officer_Tax_ID	PF_lname_fname	FS_lname_fname	Last_Name_First_Name
0	953430	SORECA, NICHOLAS V	SORECA, NICHOLAS V	SORECA, NICHOLAS
2	961465	WYNNE, JAMES W	WYNNE, JAMES W	WYNNE, JAMES
3	964085	KENJESKY, MICHAEL V	KENJESKY, MICHAEL V	KENJESKY, MICHAEL
4	964778	SINTO, MICHAEL G	SINTO, MICHAEL G	SINTO, MICHAEL
5	963591	KENNEDY, ERIN L	KENNEDY, ERIN L	KENNEDY, ERIN
...	...	...	...	...
4657	952535	CALDER, DIANDRE D	CALDER, DIANDRE D	CALDER, DIANDRE
4658	952535	CALDER, DIANDRE D	CALDER, DIANDRE D	CALDER, DIANDRE
4659	953330	RODRIGUEZ, ANDY J	RODRIGUEZ, ANDY J	RODRIGUEZ, ANDY
4660	948461	BLACKMAN, JUSTIN F	BLACKMAN, JUSTIN F	BLACKMAN, JUSTIN
4661	948461	BLACKMAN, JUSTIN F	BLACKMAN, JUSTIN F	BLACKMAN, JUSTIN



3621 rows x 4 columns

Next steps:

[Generate code with name_discrepancies](#)
[View recommended plots](#)
[New interactive sheet](#)

```
import re
```

```
def strip_middle_initials(name):
    if name == ".":
        return name
    # Convert to uppercase, remove extra spaces, then remove middle initials (assumed to be single-letter middle)
    name = name.upper().strip()
    # Remove middle initials using regex (e.g., "DOE, JOHN M" -> "DOE, JOHN")
    return re.sub(r"\b([A-Z])\b", "", name).replace(" ", " ").strip()
```

```
# Apply the cleaning function
for col in ["PF_lname_fname", "FS_lname_fname", "Last_Name_First_Name"]:
    draft_demographic_merge[f"{col}_cleaned"] = draft_demographic_merge[col].apply(strip_middle_initials)
```

```
# Check for matches again
draft_demographic_merge["name_match_no_mi"] = (
    (draft_demographic_merge["PF_lname_fname_cleaned"] == draft_demographic_merge["FS_lname_fname_cleaned"]) &
    (draft_demographic_merge["PF_lname_fname_cleaned"] == draft_demographic_merge["Last_Name_First_Name_cleaned"])
)
```

```
# Summary output
total = draft_demographic_merge.shape[0]
match = draft_demographic_merge["name_match_no_mi"].sum()
print(f"{match} of {total} rows match across all three columns after removing middle initials.")
```



4159 of 4663 rows match across all three columns after removing middle initials.

```
# Show rows where names don't match across all three cleaned columns
mismatched_names = draft_demographic_merge[draft_demographic_merge["name_match_no_mi"] == False]
```

```
# Display key columns for review
```

```
mismatched_names[[
    "Final_Officer_Tax_ID",
    "PF_lname_fname",
    "FS_lname_fname",
    "Last_Name_First_Name"
]]
```



```

    "FS_lname_fname",
    "Last_Name_First_Name",
    "PF_lname_fname_cleaned",
    "FS_lname_fname_cleaned",
    "Last_Name_First_Name_cleaned"
  ]].head(20)

```

	Final_Officer_Tax_ID	PF_lname_fname	FS_lname_fname	Last_Name_First_Name	PF_lname_fname_cleaned	FS_
6	963656	MONACO, MARISSA L	MONACO, MARISSA L	DINIS, MARISSA	MONACO, MARISSA	
9	966375	VADALA, JOSEPH G	.	VADALA, JOSEPH	VADALA, JOSEPH	
12	966613	HARIDAS, SREEKANTH	.	HARIDAS, SREEKANTH	HARIDAS, SREEKANTH	
13	966613	HARIDAS, SREEKANTH	.	HARIDAS, SREEKANTH	HARIDAS, SREEKANTH	
14	966888	.	.	TORRES, MARITZA	.	
19	959907	VASQUEZ, LOREAL	VASQUEZ, LOREAL	VAZQUEZ, LOREAL	VASQUEZ, LOREAL	
20	960914	.	.	MCGREEVY, KEITH	.	
25	951992	MCLEOD, RUSSELL M	MCLEOD, RUSSELL M	MCLEOD, RUSSEL	MCLEOD, RUSSELL	
40	964095	KLAMBATSEN, STEVEN C	KLAMBATSEN, STEVEN C	KLAMBASTEN, STEVEN	KLAMBATSEN, STEVEN	
43	965045	DIAMOND, ADAM N	DIAMOND, ADAM N	DIAMON, ADAM	DIAMOND, ADAM	
49	966053	FIGUEROE, GENESIS	FIGUEROE, GENESIS	QUICK, GENESIS	FIGUEROE, GENESIS	
51	966762	PANORA, ROSAMARIA	PANORA, ROSAMARIA	PANORA, ROSMARIA	PANORA, ROSAMARIA	
68	968804	SHOLOMOV, RAFAEL	.	SHOLOMOV, RAFAEL	SHOLOMOV, RAFAEL	
80	957044	.	.	POLANCO, JOEL	.	
93	967662	RIOSCHALCO, TATIANA Y	RIOSCHALCO, TATIANA Y	RIOCHALCO, TATIANA	RIOSCHALCO, TATIANA	
94	967662	RIOSCHALCO, TATIANA Y	RIOSCHALCO, TATIANA Y	RIOCHALCO, TATIANA	RIOSCHALCO, TATIANA	
108	957091	.	.	RODRIGUEZ, EDUARDO	.	
116	923153	.	.	SEFTON, RYAN	.	
141	960880	MARTIN, JEFFREY J	MARTIN, JEFFREY J	MARTIN, JEFFERY	MARTIN, JEFFREY	
153	927362	PITTS, BYRAN A	PITTS, BYRAN A	PITTS, BRYAN	PITTS, BYRAN	

```

# Show mismatches between cleaned name columns
mismatched_clean_names = draft_demographic_merge[
    ~(
        (draft_demographic_merge["PF_lname_fname_cleaned"] == draft_demographic_merge["FS_lname_fname_cleaned"])
        (draft_demographic_merge["PF_lname_fname_cleaned"] == draft_demographic_merge["Last_Name_First_Name_cleaned"])
        (draft_demographic_merge["FS_lname_fname_cleaned"] == draft_demographic_merge["Last_Name_First_Name_cleaned"])
    )
]

```

```
# Display results
mismatched_clean_names[
    [
        "Final_Officer_Tax_ID",
        "PF_lname_fname_cleaned",
        "FS_lname_fname_cleaned",
        "Last_Name_First_Name_cleaned"
    ]
].head(20)
```

	Final_Officer_Tax_ID	PF_lname_fname_cleaned	FS_lname_fname_cleaned	Last_Name_First_Name_cleaned
6	963656	MONACO, MARISSA	MONACO, MARISSA	DINIS, MARISSA
9	966375	VADALA, JOSEPH	.	VADALA, JOSEPH
12	966613	HARIDAS, SREEKANTH	.	HARIDAS, SREEKANTH
13	966613	HARIDAS, SREEKANTH	.	HARIDAS, SREEKANTH
14	966666	.	.	TORRES, MARITZA
19	959907	VASQUEZ, LOREAL	VASQUEZ, LOREAL	VAZQUEZ, LOREAL
20	960914	.	.	MCGREEVY, KEITH
25	951992	MCLEOD, RUSSELL	MCLEOD, RUSSELL	MCLEOD, RUSSEL
40	964095	KLAMBATSEN, STEVEN	KLAMBATSEN, STEVEN	KLAMBASTEN, STEVEN
43	965045	DIAMOND, ADAM	DIAMOND, ADAM	DIAMON, ADAM
49	966053	FIGUERO, GENESIS	FIGUERO, GENESIS	QUICK, GENESIS
51	966762	PANORA, ROSAMARIA	PANORA, ROSAMARIA	PANORA, ROSMARIA
68	968804	SHOLOMOV, RAFAEL	.	SHOLOMOV, RAFAEL
80	957044	.	.	POLANCO, JOEL
93	967662	RIOSCHALCO, TATIANA	RIOSCHALCO, TATIANA	RIOCHALCO, TATIANA
94	967662	RIOSCHALCO, TATIANA	RIOSCHALCO, TATIANA	RIOCHALCO, TATIANA
108	957091	.	.	RODRIGUEZ, EDUARDO
116	923153	.	.	SEFTON, RYAN
141	960880	MARTIN, JEFFREY	MARTIN, JEFFREY	MARTIN, JEFFERY
153	927362	PITTS, BYRAN	PITTS, BYRAN	PITTS, BRYAN

```
def confirm_fs_name(row):
    fs = row["FS_lname_fname_cleaned"]
    pf = row["PF_lname_fname_cleaned"]
    ln = row["Last_Name_First_Name_cleaned"]

    if fs == pf == ln and fs != ".":
        return fs
    elif fs == pf and fs != ".":
        return fs
    elif fs == ln and fs != ".":
        return fs
    else:
        return "."

# Create the new column
draft_demographic_merge["FS_confirmed_lnamefname"] = draft_demographic_merge.apply(confirm_fs_name, axis=1)
```

```
# Drop PF_lname_fname_cleaned
```

```
draft_demographic_merge["FS_confirmed_lnamefname"].head(20)
```



	FS_confirmed_lnamefname
0	SORECA, NICHOLAS
1	ENNIS, PATRICIA
2	WYNNE, JAMES
3	KENJESKY, MICHAEL
4	SINTO, MICHAEL
5	KENNEDY, ERIN
6	MONACO, MARISSA
7	JONES, FREDDY
8	VELAZQUEZ, ANTHONY
9	.
10	JIMENEZ, JOSE
11	KHALIQUE, MUHAMMAD
12	.
13	.
14	.
15	SINMAZISIK, NECIBE
16	ANTOINE, GREGORY
17	DRISCOLL, TROY
18	MUCCIACCIO, DANIEL
19	VASQUEZ, LOREAL

dtype: object

✓ DA, CC, AA Case Num Cleaning

- Replacing all [NaN] case insensitive to "."
- Compare across all three to check if there is consistency (these are unique case types, so these remain)
 - DA_casenum
 - CC_casenum
 - AD_allegation_case_num
- Drop DA_X8: consistent through out, may be when data was pulled or inputted into NYPD system

```
# Removing all [NaN]
```

```
cols_to_clean = [col for col in draft_demographic_merge.columns if col.startswith("CC_") or col.startswith("AD_
```

```
def replace_nan_list(val):
```

```
    # Replace lists containing only NaN with "."
```

```
    if isinstance(val, list) and all(pd.isna(v) for v in val):
```

```
        return "."
```

```
    return val
```

```
for col in cols_to_clean:
```

```
draft_demographic_merge[["Final_Officer_Tax_ID", "DA_casenum", "CC_casenum", "AD_allegation_casenum"]].head(500)
```



	Final_Officer_Tax_ID	DA_casenum	CC_casenum	AD_allegation_casenum	
0	953430	.	.	.	
1	961743	.	.	.	
2	961465	.	.	.	
3	964085	.	.	.	
4	964778	.	.	.	
...	...	...	...	...	
495	964971	.	.	.	
496	953226	.	.	.	
497	966617	.	.	[2022-25329]	
498	968338	.	.	.	
499	964354	.	.	.	

500 rows x 4 columns

```
draft_demographic_merge[["DA_casenum", "CC_casenum", "AD_allegation_casenum"]].head(20)
```



	DA_casenum	CC_casenum	AD_allegation_casenum	
0	.	.	.	
1	.	.	.	
2	.	.	.	
3	.	.	.	
4	.	.	.	
5	.	.	.	
6	.	.	.	
7	.	.	.	
8	.	.	.	
9	.	.	.	
10	.	.	.	
11	.	.	.	
12	.	.	.	
13	.	.	.	
14	.	.	.	
15	.	.	.	
16	.	.	.	
17	.	.	.	
18	.	.	.	
19	.	.	.	

```
# Do these vary?
def has_valid_value(x):
    if isinstance(x, list):
        return any(item != "." and pd.notna(item) for item in x)
    return x != "." and pd.notna(x)

cols = ["DA_casenum", "CC_casenum", "AD_allegation_casenum"]

valid_rows = draft_demographic_merge[
    draft_demographic_merge[cols].apply(lambda row: any(has_valid_value(x) for x in row), axis=1)
]

# Preview result (you can change to .head() if needed)
valid_rows[["Final_Officer_Tax_ID"] + cols]
```



	Final_Officer_Tax_ID	DA_casenum	CC_casenum	AD_allegation_casenum
31	968548	.	.	[2021-23085, 2021-23085]
32	968548	.	.	[2021-23085, 2021-23085]
90	964803	.	.	[2022-24823]
113	952486	.	[2014-12689, 2014-12689, 2014-12689]	.
129	946446	.	[2010-1132]	.
...	...	...	...	...
4495	963589	.	.	[2020-22294]
4508	966086	.	.	[2020-22990]
4538	953658	.	[2020-21828, 2020-21828, 2020-21828, 2020-2182...	[2021-24029]
4622	954931	.	.	[2019-21404]
4644	963691	.	[2018-18825, 2018-18825]	.

239 rows x 4 columns




```
# Checking overlap between DA_casenum and CC_casenum
overlap_rows = draft_demographic_merge[
    draft_demographic_merge.apply(
        lambda row: bool(
            set(str(row["DA_casenum"]).split(",")) & set(str(row["CC_casenum"]).split(","))
        ) if row["DA_casenum"] != "." and row["CC_casenum"] != "." else False,
        axis=1
    )
]

overlap_rows[
    "Final_Officer_Tax_ID",
    "DA_casenum", "DA_disposition", "DA_charge",
    "CC_casenum", "CC_disposition", "CC_charge"
]

# 18/19 of the DA overlap with CC: use
```



	Final_Officer_Tax_ID	DA_casenum	DA_disposition	DA_charge	CC_casenum	CC_disposition	CC_c
528	955104	[2018-19106]	[GUILTY]	[Wrongful used physical force.]	[2018-19106]	[GUILTY]	[Wrongful physical
540	945080	[2018-18855]	[GUILTY]	[WHILE ON-DUTY WAS DISCOURTEOUS, IN THAT HE SP...	[2018-18855]	[GUILTY]	[WHILE ON DISCOURT IN THAT H
664	939265	[2015-14005, 2015-14005, 2015-14005, 2015-1400...	[GUILTY, GUILTY, GUILTY, GUILTY]	[WHILE ON-DUTY, WAS DISCOURTEOUS TO A PERSON K...	[2015-14005, 2015-14005, 2015-14005, 2015-1400...	[GUILTY, GUILTY, GUILTY, GUILTY]	[WHIL DUTY DISCOURT TO A PE
774	949807	[2016-16075]	[GUILTY]	[WHILE ON-DUTY, DID WRONGFULLY UTILIZE THE DEP...	[2016-16075]	[GUILTY]	[WHIL DUTY WRONG UTILIZ
1044	938645	[2018-18475, 2018-18475, 2018-18475, 2018-1847...	[GUILTY, GUILTY, GUILTY, GUILTY]	[while on-duty, did wrongfully and without jus...	[2018-18475, 2018-18475, 2018-18475, 2018-1847...	[GUILTY, GUILTY, GUILTY, GUILTY]	[while on-di wrongfu withoi
1263	932277	[2017-17814]	[GUILTY]	[ON DUTY, ABUSED HIS AUTHORITY, IN THAT HE FRIS...	[2017-17814]	[GUILTY]	DUTY, AE HIS AUTHI IN TH
3203	916969	[2015-13446, 2015-13446, 2015-13446]	[GUILTY, GUILTY, GUILTY]	[WHILE ON-DUTY, STOPPED A PERSON KNOWN TO THE ...	[2015-13446, 2015-13446, 2015-13446, 2017-17036]	[GUILTY, GUILTY, GUILTY, GUILTY]	[WHIL DUTY, STC A PE KNOWN T
3204	941041	[2016-15849, 2016-15849, 2016-15849]	[PLEADED GUILTY, PLEADED GUILTY, PLEADED GUILTY]	[WHILE ON-DUTY, WRONGFULLY USED HIS CELLULAR T...	[2016-15849, 2016-15849, 2016-15849]	[PLEADED GUILTY, PLEADED GUILTY, PLEADED GUILTY]	[WHIL WRONG USE CELLUL
3262	938629	[2015-14337, 2015-14337]	[GUILTY, GUILTY]	[WHILE ON-DUTY, WAS DISCOURTEOUS TO ON-DUTY, N...	[2015-14337, 2015-14337]	[GUILTY, GUILTY]	[WHIL DUTY DISCOURT TO ON-DU
3667	947051	[2016-16228]	[GUILTY]	[WHILE ON-DUTY, FAILED TO CONDUCT A THOROUGH F...	[2016-16228]	[GUILTY]	[WHIL DUTY, FAIL COND THOROU
3717	953225	[2015-14033, 2015-14033, 2017-17118, 2017-1711...	[PLEADED GUILTY, PLEADED GUILTY, PLEADED GUILTY]	[WRONGFULLY USED FORCE, IN THAT WITHOUT POLICE...	[2015-14033, 2017-17118, 2017-17118, 2017-18103]	[PLEADED GUILTY, PLEADED GUILTY, PLEADED GUILTY]	[WRONG USED FOR THAT WIT PO
3752	935629	[2017-18259, 2017-18259, 2017-18259, 2017-1825...	[GUILTY, GUILTY, GUILTY, GUILTY]	[FAILED TO REPORT TO DUTY IN A TIMELY MANNER F...	[2017-18259, 2017-18259, 2017-18259, 2017-1825...	[GUILTY, GUILTY, GUILTY, GUILTY]	[FAIL REPO DUTY TIMELY MA
3819	938645	[2018-18475, 2018-18475, 2018-18475, 2018-1847...	[GUILTY, GUILTY, GUILTY, GUILTY]	[while on-duty, did wrongfully and without jus...	[2018-18475, 2018-18475, 2018-18475, 2018-1847...	[GUILTY, GUILTY, GUILTY, GUILTY]	[while on-di wrongfu withoi
		[2015-12955,	[GUILTY, GUILTY]	[WHILE ON-DUTY, ABUSED	[2015-12955,	[GUILTY, GUILTY]	[WHIL DUTY, AE

3831	955270	2015-12955, 2015-12955]	[GUILTY, GUILTY, GUILTY]	DUTY, ABUSED HIS AUTHORITY IN THAT H...	2015-12955, 2015-12955]	[GUILTY, GUILTY, GUILTY]	DUTY, AB HIS AUTH IN TH
3890	940354	[2015-13772, 2015-13772, 2015-13772]	[GUILTY, GUILTY, GUILTY]	[WHILE ON- DUTY, FAILED TO REQUEST THE RESPONSE...	[2015-13772, 2015-13772, 2015-13772]	[GUILTY, GUILTY, GUILTY]	[WHIL DUTY, FAIL REQUES RESPC
4024	948300	[2017-18247, 2017-18247]	[PLEADED GUILTY, PLEADED GUILTY]	[DID KNOWINGLY ASSOCIATE WITH PERSONS OR ORGAN...	[2017-18247, 2017-18247]	[PLEADED GUILTY, PLEADED GUILTY]	[DID KNOW ASSC WITH PEF OR OR
4143	941041	[2016-15849, 2016-15849, 2016-15849]	[PLEADED GUILTY, PLEADED GUILTY, PLEADED GUILTY]	[WHILE ON- DUTY, WRONGFULLY USED HIS CELLULAR T...	[2016-15849, 2016-15849, 2016-15849]	[PLEADED GUILTY, PLEADED GUILTY, PLEADED GUILTY]	[WHIL WRONG USE CELLUL
4271	938645	[2018-18475, 2018-18475, 2018-18475, 2018-1847...	[GUILTY, GUILTY, GUILTY, GUILTY, GUILTY]	[while on-duty, did wrongfully and without jus...	[2018-18475, 2018-18475, 2018-18475, 2018-1847...	[GUILTY, GUILTY, GUILTY, GUILTY, GUILTY]	[while on-di wrongfu withoi

```
# Cleaning the overlapping rows
matching = overlap_rows[
    overlap_rows["DA_casenum"] == overlap_rows["CC_casenum"]
]

# Create cleaned columns
matching["casenum_clean"] = matching["DA_casenum"]
matching["disposition_clean"] = matching["DA_disposition"] # or CC if preferred
matching["charge_clean"] = matching["DA_charge"] # or CC if preferred

# Drop redundant fields
cleaned = matching[[
    "Final_Officer_Tax_ID", "casenum_clean", "disposition_clean", "charge_clean"
]]

# Flag if DA and CC casenums are identical
draft_demographic_merge["casenum_match"] = (
    draft_demographic_merge["DA_casenum"] == draft_demographic_merge["CC_casenum"]
)

def consolidate_case(row):
    if row["casenum_match"]:
        return {
            "casenum_clean": row["DA_casenum"],
            "disposition_clean": row["DA_disposition"],
            "charge_clean": row["DA_charge"]
        }
    else:
        # If mismatched, join both sets with a delimiter or keep them separately
        return {
            "casenum_clean": f"{row['DA_casenum']} / {row['CC_casenum']}",
            "disposition_clean": f"{row['DA_disposition']} / {row['CC_disposition']}",
            "charge_clean": f"{row['DA_charge']} / {row['CC_charge']}"
        }

# Apply
consolidated = draft_demographic_merge.apply(consolidate_case, axis=1, result_type='expand')
draft_demographic_merge["CCDA_casenum"] = consolidated["casenum_clean"]
draft_demographic_merge["CCDA_disposition"] = consolidated["disposition_clean"]
```

```
draft_demographic_merge["CCDA_charge"] = consolidated["charge_clean"]
```

```
print(f"Total overlapping case rows: {overlap_rows.shape[0]}")
```

```
↗ Total overlapping case rows: 18
```

```
draft_demographic_merge[[
    "Final_Officer_Tax_ID",
    "CCDA_casenum",
    "CCDA_disposition",
    "CCDA_charge"
]].head(10)
```

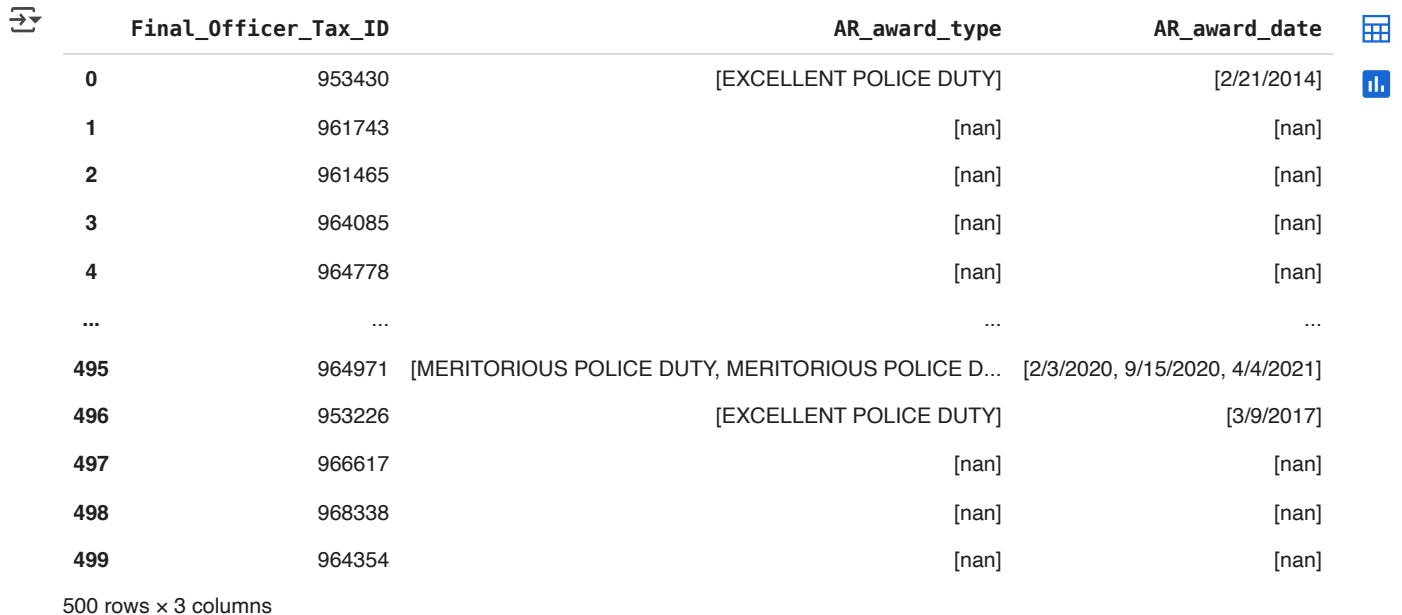
↗

	Final_Officer_Tax_ID	CCDA_casenum	CCDA_disposition	CCDA_charge	
0	953430	.	.	.	
1	961743	.	.	.	
2	961465	.	.	.	
3	964085	.	.	.	
4	964778	.	.	.	
5	963591	.	.	.	
6	963656	.	.	.	
7	952904	.	.	.	
8	966902	.	.	.	
9	966375	.	.	.	

✓ AR Award Summary

- Grouping award type and creating new variable (award_summary)
- Drop NaNs

```
draft_demographic_merge[[
    "Final_Officer_Tax_ID",
    "AR_award_type",
    "AR_award_date"
]].head(500)
```

The table displays officer information with columns: Final_Officer_Tax_ID, AR_award_type, and AR_award_date. It shows a list of officers with their respective award types and dates. The table is truncated, showing rows 0 through 499.

	Final_Officer_Tax_ID	AR_award_type	AR_award_date
0	953430	[EXCELLENT POLICE DUTY]	[2/21/2014]
1	961743	[nan]	[nan]
2	961465	[nan]	[nan]
3	964085	[nan]	[nan]
4	964778	[nan]	[nan]
...	...	...	...
495	964971	[MERITORIOUS POLICE DUTY, MERITORIOUS POLICE D...	[2/3/2020, 9/15/2020, 4/4/2021]
496	953226	[EXCELLENT POLICE DUTY]	[3/9/2017]
497	966617	[nan]	[nan]
498	968338	[nan]	[nan]
499	964354	[nan]	[nan]

500 rows x 3 columns

```
from collections import Counter
```

```
def award_summary(row):
    awards = row["AR_award_type"]

    # Case 1: it's a list with all NaNs or a single NaN
    if isinstance(awards, list):
        # Flatten and clean: remove NaNs, empty strings, "."
        clean_awards = [a.strip().upper() for a in awards if isinstance(a, str) and a.strip() != "."]
        if not clean_awards:
            return "."
        counts = Counter(clean_awards)
        return " / ".join([f"{count} {award}" for award, count in counts.items()])

    # Case 2: it's a string
    if isinstance(awards, str):
        if awards.strip() == "." or awards.strip().lower() == "nan":
            return "."
        clean_awards = [a.strip().upper() for a in awards.split(",") if a.strip()]
        if not clean_awards:
            return "."
        counts = Counter(clean_awards)
        return " / ".join([f"{count} {award}" for award, count in counts.items()])

    # Case 3: it's NaN or not interpretable
    return "."
```

```
# Apply the function
draft_demographic_merge["AR_award_summary"] = draft_demographic_merge.apply(award_summary, axis=1)
```

```
#draft_demographic_merge = draft_demographic_merge.fillna(".")
```

```
draft_demographic_merge[[
    "Final_Officer_Tax_ID",
    "AR_award_type",
    "AR_award_date",
    "AR_award_summary"
]].head(500)
```



Final_Officer_Tax_ID		AR_award_type	AR_award_date	AR_award_summary	
0	953430	[EXCELLENT POLICE DUTY]	[2/21/2014]	1 EXCELLENT POLICE DUTY	
1	961743	[nan]	[nan]	.	
2	961465	[nan]	[nan]	.	
3	964085	[nan]	[nan]	.	
4	964778	[nan]	[nan]	.	
...	...	...	...	...	
495	964971	[MERITORIOUS POLICE DUTY, MERITORIOUS POLICE D...	[2/3/2020, 9/15/2020, 4/4/2021]	3 MERITORIOUS POLICE DUTY	
496	953226	[EXCELLENT POLICE DUTY]	[3/9/2017]	1 EXCELLENT POLICE DUTY	
497	966617	[nan]	[nan]	.	
498	968338	[nan]	[nan]	.	
499	964354	[nan]	[nan]	.	

500 rows x 4 columns

```
draft_demographic_merge[[
  "Final_Officer_Tax_ID",
  "AR_award_type",
  "AR_award_date",
  "AR_award_summary"
]].head(500)
```



Final_Officer_Tax_ID		AR_award_type	AR_award_date	AR_award_summary	
0	953430	[EXCELLENT POLICE DUTY]	[2/21/2014]	1 EXCELLENT POLICE DUTY	
1	961743	[nan]	[nan]	.	
2	961465	[nan]	[nan]	.	
3	964085	[nan]	[nan]	.	
4	964778	[nan]	[nan]	.	
...	...	...	...	...	
495	964971	[MERITORIOUS POLICE DUTY, MERITORIOUS POLICE D...	[2/3/2020, 9/15/2020, 4/4/2021]	3 MERITORIOUS POLICE DUTY	
496	953226	[EXCELLENT POLICE DUTY]	[3/9/2017]	1 EXCELLENT POLICE DUTY	
497	966617	[nan]	[nan]	.	
498	968338	[nan]	[nan]	.	
499	964354	[nan]	[nan]	.	

500 rows x 4 columns

✓ Cleaning up final dataset

- Replacing all NaNs
- Dropping unneeded columns